



Wydział Informatyki i Telekomunikacji
Kierunek studiów: Informatyka techniczna
Specjalność: Grafika i systemy multimedialne
Rok akademicki: 2024/2025

RAPORT KOŃCOWY - WŁASNA GRA

Patrycja Smits, 272940
Julia Krok, 272981

Projekt
Projektowanie i programowanie gier
Wtorek (tydzień parzysty) 13:15

Prowadzący:
dr inż. arch. Tomasz Zamojski

Spis treści

1	Wstęp	3
2	Opis rozgrywki	3
2.1	Mechanika gry	3
2.1.1	Ruch i interakcja	3
2.1.2	Uprawa roślin	3
2.1.3	System pogody	3
2.1.4	Sklep i ekonomia	3
2.1.5	Zarządzanie zasobami	3
2.1.6	Interfejs użytkownika	4
2.2	Postać gracza	4
2.3	Samouczek	4
2.4	System zadań	5
3	Interfejs użytkownika	6
3.1	Mapa gry	6
3.1.1	Tworzenie i struktura Tilesetów	6
3.2	Ekran startowy	8
3.3	Elementy wyświetlane na ekranie (HUD)	8
3.4	Widok sklepu	9
4	Opis techniczny	10
4.1	Architektura projektu	10
4.2	Maszyna stanów gracza	11
4.2.1	Cel zastosowania maszyny stanów	11
4.2.2	Architektura maszyny stanów	12
4.2.3	Przykład przejścia między stanami	13
4.2.4	Integracja z innymi systemami	13
4.3	System kamery	13
4.3.1	Zasada działania	13
4.3.2	Integracja z hierarchią sceny	13
4.3.3	Optymalizacja pod grafikę pikselową	14
4.3.4	Zalety zastosowanego rozwiązania	14
4.4	System używania narzędzi	14
4.4.1	Komponenty i ich role	14
4.4.2	Struktura obiektów interaktywnych	16
4.4.3	Mechanika działania	17
4.4.4	Zalety systemu	17
4.5	System czasu	17
4.6	System pogody	18
4.7	System wzrostu roślin	18
4.8	System sadzenia roślin	22
4.9	System ekwipunku, paska narzędzi i skrzyń	23
4.9.1	Pasek narzędzi (ToolsPanel)	24
4.9.2	Ekwipunek gracza	26
4.9.3	Skrzynie i przechowywanie przedmiotów	28
4.10	System samouczka	31
4.10.1	Uruchamianie samouczka i interakcja z przewodnikiem	31
4.10.2	Struktura hierarchii UI	32
4.10.3	Skrypt <code>game_dialogue_balloon.gd</code> – logika działania	33
4.10.4	Plik dialogowy – struktura i działanie	34
4.10.5	GameState i mechanizm pauz w samouczku	34

4.10.6	Elementy wizualne i animacje	35
4.11	System zadań	36
4.11.1	Struktura	36
4.11.2	Wybrane funkcje systemu	36
4.11.3	Kolejność i aktualizacja	36
4.11.4	Interfejs użytkownika	37
4.11.5	Integracja z grą	37
4.12	System działania sklepu	37
4.12.1	Obecność w grze	37
4.12.2	Struktura UI	38
4.12.3	System sprzedaży przedmiotów	39
5	Opis napotkanych problemów	40
5.1	Dźwięk podczas zbierania collectables	40
5.2	Wyświetlanie elementów	41
5.3	Problemy z integracją ekwipunku i panelu narzędzi	42
5.3.1	Refaktoryzacja struktury danych	42
5.3.2	Efekty wprowadzenia jednolitego modelu	42
6	Podsumowanie	43
7	Źródła	44

1 Wstęp

Celem niniejszego projektu było stworzenie gry 2D z elementami symulacji uprawy roślin oraz prostym systemem ekonomicznym. Projekt został zrealizowany w silniku *Godot Engine* z wykorzystaniem języka *GScript*. W ramach projektu powstała gra „*Bloomrise*”. Gra została zaprojektowana jako samodzielna aplikacja, w której gracz wciela się w postać kota-ogrodnika, który uprawia różne gatunki kwiatów, sprzedaje je w sklepie i zarządza posiadanymi zasobami.

W trakcie prac nad grą skupiono się na implementacji takich mechanik jak: interakcja z otoczeniem za pomocą narzędzi (kilof, siekiera, motyka, konewka), system sadzenia i wzrostu roślin, system pogody (m.in. deszcz wpływający na rozgrywkę), interfejs sklepu oraz zarządzanie ekwipunkiem gracza. Istotnym celem było również opracowanie spójnego i responsywnego interfejsu użytkownika oraz zapewnienie podstawowych efektów dźwiękowych i wizualnych.

Gra ma charakter otwarty, pozwalając graczowi na swobodną eksplorację i rozwój ogrodu. Projekt stanowił ćwiczenie zarówno z zakresu projektowania gier, jak i praktycznego wykorzystania silnika Godot w bardziej złożonych produkcjach.

Kod źródłowy gry dostępny jest publicznie w repozytorium GitHub: <https://github.com/Ellemire/Bloomrise>

2 Opis rozgrywki

2.1 Mechanika gry

Gra opiera się na prostych mechanikach rolniczych i ekonomicznych, które razem tworzą zamknięty cykl rozgrywki. Gracz przeprowadzany jest przez mechanikę gry przez sowę-przewodnika o imieniu *Wisowl*. Główne elementy mechaniki wraz z opisami przedstawiono w kolejnych punktach.

2.1.1 Ruch i interakcja

Gracz steruje postacią ogrodnika poruszającego się po dwuwymiarowej mapie. Postać może wchodzić w interakcję z elementami otoczenia, takimi jak kamienie, drzewa, krawędzie mapy, przewodnik czy sklep. Interakcje są aktywowane poprzez użycie klawiatury lub myszy.

2.1.2 Uprawa roślin

Gracz może używać motyki do przygotowania ziemi oraz sadzić nasiona roślin. Rośliny przechodzą przez cztery fazy wzrostu, a po osiągnięciu dojrzałości zostają zebrane. Każda faza wzrostu trwa przynajmniej dwa dni. Aby przejść do kolejnej fazy, roślina musi zostać podlana.

2.1.3 System pogody

Gra zawiera prosty system pogodowy, który wpływa na uprawy. Przykładowo, podczas deszczu nie ma potrzeby ręcznego podlewania roślin, co zwiększa dynamikę i urozmaica rozgrywkę. Efektowi towarzyszą animacje oraz dźwięki.

2.1.4 Sklep i ekonomia

Zebrane zasoby można sprzedać w sklepie w zamian za monety. Gracz może również kupować nasiona nowych kwiatów. Ceny i dostępność przedmiotów są prezentowane w graficznym interfejsie z przyciskami do kupna i sprzedaży.

2.1.5 Zarządzanie zasobami

Gracz zarządza swoim ekwipunkiem, zawierającym nasiona i inne przedmioty. Dodatkowo ma ograniczoną ilość złota, które musi wydawać rozsądnie, by zwiększyć swoje zyski w przyszłości. Otrzymane przedmioty domyślnie przechowywane są w inwentarzu o ograniczonej pojemności, w przypadku wypełnienia inwentarza może przenieść część przedmiotów do skrzyni znajdującej się w domu.

2.1.6 Interfejs użytkownika

Na ekranie widoczne są kluczowe informacje, takie jak liczba posiadanych monet, ekwipunek i aktualne działania. Interfejs jest dostosowany do dynamiki gry i pozwala na intuicyjne zarządzanie zasobami.

2.2 Postać gracza

Gracz wciela się w rolę kota-ogrodnika, którego zadaniem jest rozwijanie własnego ogrodu poprzez uprawę kwiatów, zarządzanie zasobami oraz prowadzenie prostych transakcji handlowych. Postać porusza się po świecie gry w czterech kierunkach (góra, dół, lewo, prawo) i może wchodzić w interakcje z otoczeniem za pomocą przypisanych narzędzi i przycisków.

Każda akcja gracza – poruszanie się, ścinanie drzew, wydobywanie kamieni, przygotowywanie pola, sadzenie nasion, podlewanie kwiatów – jest wspierana animacjami i reakcjami wizualnymi. Kierunek, w którym zwrócona jest postać, wpływa na miejsce wykonania akcji (np. kopanie odbywa się przed postacią, zgodnie z jej orientacją).

Gracz ma do dyspozycji podstawowy interfejs HUD, który informuje o liczbie posiadanych monet oraz umożliwia dostęp do ekwipunku. W trakcie rozgrywki rozwija swój ogród, zdobywając nowe nasiona.



Rysunek 1: Zrzut ekranu pokazujący wygląd postaci gracza.

2.3 Samouczek

Rozgrywkę w *Bloomrise* rozpoczyna interaktywny samouczek prowadzony przez sowę o imieniu **Wisowl**, która pełni funkcję przewodnika i narratora. Samouczek opiera się na sekwencji dialogów, w których gracz podejmuje decyzje, a gra dynamicznie reaguje na jego postępy. Wisowl prowadzi gracza krok po kroku przez najważniejsze mechaniki gry.

Na początku gry pojawia się pytanie:

„Hi and welcome to Bloomrise! I’m Wisowl, your guide and companion on this wonderful journey. Before you start, would you like a quick tutorial?”

Gracz może wybrać jedną z dwóch opcji:

- **Tak, poproszę** – rozpoczyna samouczek,

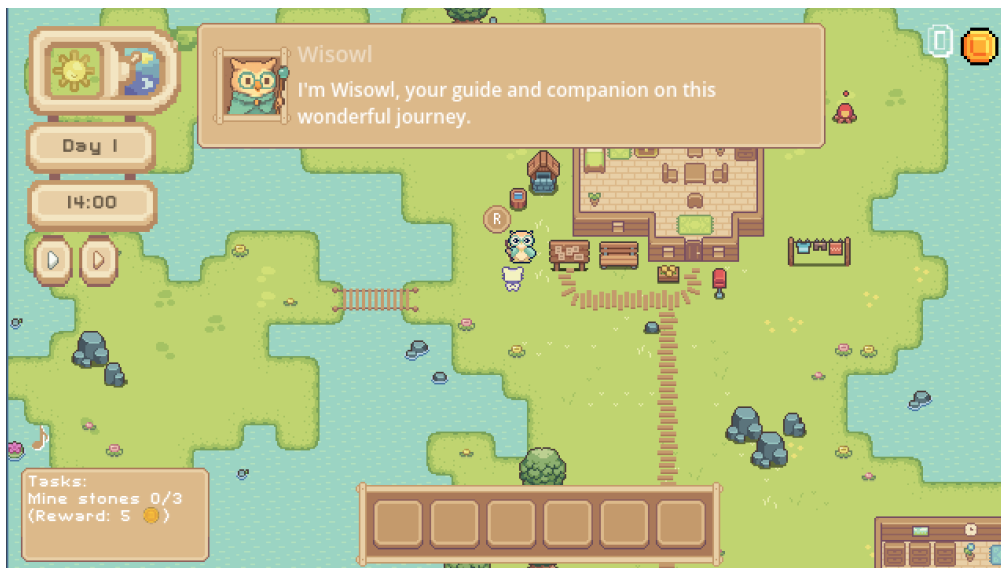
- **Nie, wszystko już wiem** – pomija samouczek, ale gracz otrzymuje wszystkie podstawowe narzędzia i nasiona.

Przebieg samouczka:

1. **Zbieranie surowców:** Gracz otrzymuje kilof i siekierę. Uczy się, jak korzystać z paska narzędzi oraz jak zbierać kamienie i drewno przy pomocy przycisku Q. Materiały te można później sprzedać.
2. **Ekwipunek:** Wisowl tłumaczy, jak otworzyć ekwipunek klawiszem E, jak przenosić przedmioty między ekwipunkiem a paskiem narzędzi (LMB, RMB), a także jak wyrzucać niepotrzebne przedmioty.
3. **Skrzynia w domu:** Gracz poznaje mechanizm skrzyni domowej, do której może przenosić przedmioty przy pomocy klawisza Shift oraz LPM.
4. **Sadzenie:** Po zdobyciu podstawowych zasobów Wisowl przekazuje graczowi motykę i nasiona róż. Gracz uczy się, jak przygotować ziemię oraz jak siać rośliny. Przycisk CAPSLOCK umożliwia "usunięcie" przygotowania gleby.
5. **Podlewanie:** Gracz otrzymuje konewkę i uczy się podlewania roślin – czynności niezbędnej do ich wzrostu.
6. **Sklep:** Na koniec samouczka Wisowl informuje gracza, że może podążać ścieżką do sklepu, gdzie możliwa jest sprzedaż i zakup przedmiotów.

Po każdym etapie gracz może zdecydować, czy chce kontynuować naukę, czy potrzebuje więcej czasu. Samouczek kończy się po przejściu przez wszystkie kroki, a Wisowl żegna się słowami:

„That’s all I can teach you for now. Good luck, Bloomseeker! Your journey in Bloomrise begins!”



Rysunek 2: Wygląd samouczka

2.4 System zadań

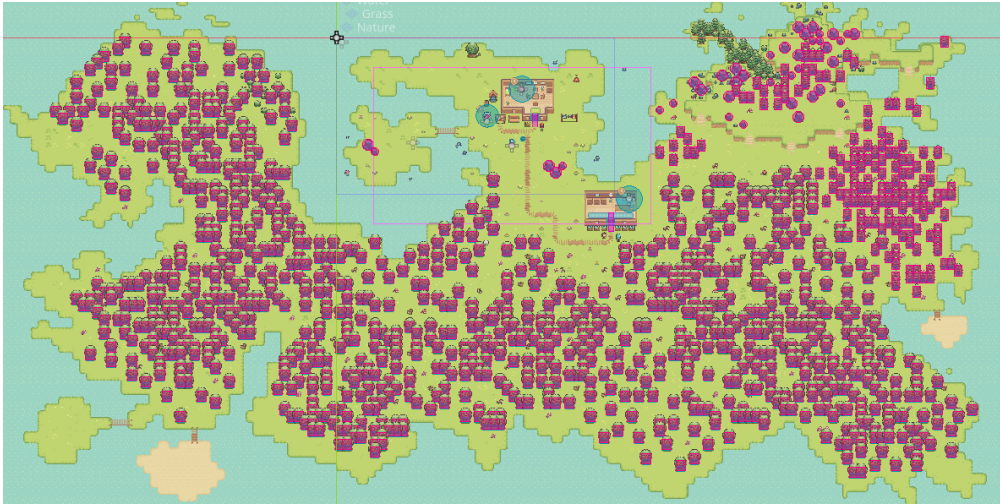
System zadań służy do dynamicznego przydzielania celów graczowi (np. zbierz kamienie, posadź kwiaty, zarób monety) i śledzenia postępu ich wykonania. Umożliwia liniowe lub warunkowe prowadzenie gracza przez samouczek oraz dalszy rozwój gry.

3 Interfejs użytkownika

Gra *Bloomrise* posiada przejrzysty, intuicyjny interfejs użytkownika, który wspiera gracza na każdym etapie rozgrywki. Składa się z kilku głównych komponentów:

3.1 Mapa gry

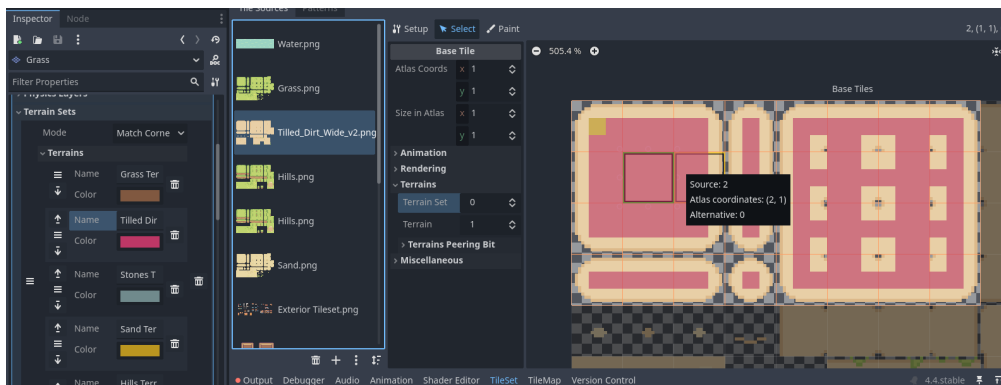
Mapa świata gry została zaprojektowana jako otwarty obszar z wyraźnie wyodrębnionymi strefami: dom gracza, ogród, ścieżki prowadzące do sklepu oraz otoczenie z interaktywnymi elementami (skały, drzewa, grządki). Mimo braku klasycznej mini-mapy, orientacja przestrzenna jest ułatwiona dzięki konsekwentnemu rozmieszczeniu obiektów i czytelnemu stylowi graficznemu.



Rysunek 3: Mapa świata gry

3.1.1 Tworzenie i struktura Tilesetów

W projekcie wykorzystano kilka niezależnych **TileSet**ów, które zostały przygotowane i skonfigurowane w edytorze Godota na potrzeby różnych warstw środowiska gry. Każdy **TileSet** posiada unikalne właściwości graficzne oraz funkcjonalne — takie jak kolizje, animacje czy reakcje na interakcje gracza.



Rysunek 4: Interfejs tworzenia Tilesetów

1. `game_tile_set.tres`

- **Zawartość:** trawa (`grass`), wzgórza (`hills`), piasek (`sand`), woda (`water`), oraz gleba uprawna (`tilled_dirt`).

- **Animacje:** kafelek `water` posiada sekwencję animacji (4 klatki) przypisaną jako `AnimatedTile`, ustawioną w edytorze z odpowiednim tempem.
- **Funkcje specjalne:**
 - `tilled.dirt` jest aktualizowany dynamicznie, gdy gracz użyje motyki — zmiana jest dokonywana za pomocą kodu.
 - Tiles posiadają kolizję tylko tam, gdzie jest to wymagane (np. wzgórze).

2. `house_tile_set.tres`

- **Zawartość:** kafle ścian, podłóg, mebli oraz animowane drzwi.
- **Animacje drzwi:** drzwi są zdefiniowane jako `AnimatedTile`, z klatkami odpowiadającymi stanom otwarcia. Stan drzwi jest modyfikowany z kodu poprzez zmianę animacji.
- **Kolizje:** ściany i meble posiadają odpowiednio zdefiniowane `CollisionShape2D`, które blokują ruch gracza.

```

1 func _ready() -> void:
2     interactable_component.interactable_activated.connect(on_interactable_activated)
3     interactable_component.interactable_deactivated.connect(on_interactable_deactivated)
4     collision_layer = 1
5
6 func on_interactable_activated() -> void:
7     animated_sprite_2d.play("open_door")
8     collision_layer = 2
9     open_door_sound.play()
10
11 func on_interactable_deactivated() -> void:
12     animated_sprite_2d.play("close_door")
13     collision_layer = 1
14     close_door_sound.play()

```

3. `exterior_tile_set.tres`

- **Zawartość:** ognisko (`campfire`), studnia, wiaderko, pranie, inne obiekty dekoracyjne.
- **Animacje:** ognisko to animowany kafel z efektami ognia (7 klatek cykliczne).
- **Kolizje:** obiekty fizyczne (np. studnia) posiadają kształt kolizji ustawiony manualnie.

4. `trees_tile_set.tres`

- **Zawartość:** drzewa liściaste (`deciduous_tree`), drzewa iglaste (`confer_tree`), kamienie (`big_stone`, `medium_stone`).
- **Złożone kolizje i logika:** obiekty z tego zestawu nie są wyłącznie kaflami — zostały przekształcone w sceny (`.tscn`), co pozwala na obsługę interakcji.
- **Przykład interakcji:**
 - Obiekt drzewa lub kamienia posiada komponent `Area2D` do wykrywania uderzeń oraz komponent `DamageComponent`.
 - Po osiągnięciu maksymalnych obrażeń wywoływana jest funkcja `on_max_damage_reached`, która usuwa obiekt i może zainstancjonować nowy (`'stone.tscn'`):

```

1 func on_max_damage_reached() -> void:
2     call_deferred("add_stone_scene")
3     queue_free()
4
5 func add_stone_scene() -> void:
6     var stone_instance = stone_scene.instantiate()
7     stone_instance.global_position = global_position
8     get_parent().add_child(stone_instance)

```

- **Kolizje:** drzewa i kamienie posiadają przypisane CollisionShape2D).

Wspólne cechy wszystkich tilesetów

- Zostały utworzone przy pomocy edytora TileSet Editor w Godot 4.0,
- Użyto Atlas Tiles dla większej wydajności (wszystkie grafiki są częścią sprite sheetów),
- Kolizje zostały przypisane ręcznie tam, gdzie wymaga tego rozgrywka,
- Animacje utworzono poprzez definiowanie klatek w obrębie jednego sprite sheetu,
- Interaktywne tilesy (drzewa/kamienie) reprezentowane są przez odrębne sceny i nie są kafkami statycznymi.

3.2 Ekran startowy

Ekran tytułowy gry jest prosty i estetyczny. Zawiera przycisk do rozpoczęcia gry oraz opcje wyjścia. W tle znajduje się statyczna ilustracja przedstawiająca głównego bohatera podczas opieki nad ogrodem.



Rysunek 5: Ekran startowy gry

3.3 Elementy wyświetlane na ekranie (HUD)

HUD jest minimalistyczny i nie przeszkadza w rozgrywce. Składa się z następujących elementów:

- **Licznik złota** — umieszczony w prawym górnym rogu ekranu, informuje o aktualnej ilości posiadanych monet.

- **Panel pogody** — umieszczony w lewym górnym rogu ekranu, informuje o aktualnej godzinie oraz dniu. Zawiera także wskaźnik pory dnia oraz ikonę pogody. Poniżej znajdują się dwa przyciski pozwalające na przyspieszenie czasu gry.
- **Panel zadań** — umieszczony w lewym dolnym rogu ekranu, pokazuje aktywne cele gracza.
- **Przycisk wyciszenia muzyki** — umieszczony w lewym dolnym rogu ekranu, pozwala na włączenie lub wyłączenie muzyki.
- **Panel narzędzi** — umieszczony na dole ekranu, posiada sześć pól pozwalających na wybór aktualnie wykorzystywanego narzędzia.
- **Panel ekwipunku** — domyślnie niewidoczny, pojawia się po naciśnięciu przycisku E. Zawiera zebrane przedmioty, umieszczone w siatce 4×3.



Rysunek 6: HUD gry

3.4 Widok sklepu

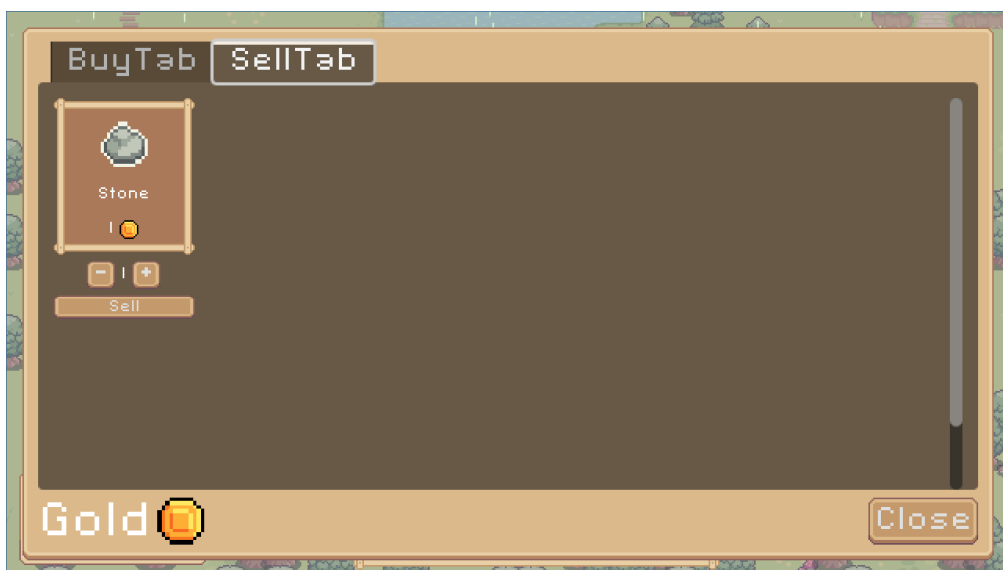
Sklep został zaprojektowany jako osobny panel (*CanvasLayer*), który pojawia się na ekranie po wejściu w interakcję z NPC lub obiektem sklepowym. Interfejs ten dzieli się na dwie zakładki:

- **Kupno (Buy)** — siatka nasion do zakupu. Każdy przedmiot ma ikonę, nazwę, etykietę z ceną oraz przycisk kupna.
- **Sprzedaż (Sell)** — dynamiczna siatka z przedmiotami z ekwipunku, które można sprzedać. Gracz może wybrać ilość przedmiotów i sprzedać je za złoto.

W dolnej części interfejsu znajduje się etykieta ze stanem konta gracza oraz przycisk zamykający interfejs sklepu.



Rysunek 7: Sekcja kupna w sklepie



Rysunek 8: Sekcja sprzedaży w sklepie

4 Opis techniczny

4.1 Architektura projektu

Gra została zbudowana w silniku Godot z wykorzystaniem paradygmatu scen i węzłów (nodes). Projekt podzielony jest na odrębne komponenty logiczne: gracz, środowisko, interfejs użytkownika oraz systemy zarządzające (m.in. czasem, pogodą, wzrostem roślin). Kluczowe systemy implementowane są jako singletony (Autoload), co umożliwia ich globalny dostęp i współdzielenie danych.

Hierarchia komponentów została logicznie uporządkowana w folderach tematycznych:

- Assets/ – elementy graficzne i dźwiękowe
- Dialogues/ – dialogi z przewodnikiem

- Inventory/ – system ekwipunku
- Scenes/ – sceny gry
 - Characters/ – skrypty i sceny postaci
 - Components/ – komponenty scen
 - Houses/ – sceny budynków
 - Level/ – główna scena gry
 - Objects/ – sceny przedmiotów
 - Test/ – sceny testowe
 - UI/ – sceny interfejsu użytkownika
- Scripts/ – skrypty
 - Globals/ – skrypty globalne
 - State_machine/ – skrypty maszyny stanów
- Tilesets/ – zestawy elementów do tworzenia mapy

4.2 Maszyna stanów gracza

Sterowanie postacią w grze *Bloomrise* zostało zaprojektowane w oparciu o elastyczną i modułarną architekturę maszyny stanów (*State Machine*). Pozwoliło to na precyzyjne odwzorowanie różnych zachowań gracza (chodzenie, bezczynność, użycie narzędzi) z pełną kontrolą nad animacjami, logiką interakcji i przejściami pomiędzy stanami.

4.2.1 Cel zastosowania maszyny stanów

Zastosowanie maszyny stanów (*State Machine*) w projekcie *Bloomrise* było świadomą decyzją projektową wynikającą z potrzeby uporządkowanego zarządzania zachowaniami postaci gracza. Tradycyjne podejście, oparte na wielu warunkach logicznych w jednym skrypcie (np. `if-else` w `_process()`), szybko staje się trudne w utrzymaniu, szczególnie w przypadku gry zawierającej liczne interakcje i animacje zależne od kontekstu.

Maszyna stanów została użyta, ponieważ umożliwia:

- **Rozdzielenie logiki** na osobne klasy – każda akcja gracza (chodzenie, bezczynność, użycie narzędzi) posiada własny, samodzielny skrypt, co zwiększa czytelność i umożliwia niezależny rozwój tych modułów.
- **Ścisłą kontrolę nad cyklem życia stanu** – każde zachowanie może mieć zdefiniowane zachowanie przy wejściu (`_on_enter()`), w czasie trwania (`_on_physics_process()`) oraz przy wyjściu (`_on_exit()`), co pozwala np. na precyzyjne włączenie kolizji podczas korzystania z narzędzi, odtworzenie dźwięku czy zakończenie animacji.
- **Uproszczenie przejść między zachowaniami** – przełączanie między stanami odbywa się przez emitowanie sygnału, dzięki czemu zmiana zachowania nie wymaga ręcznego resetowania pozostałych efektów.
- **Łatwe skalowanie systemu** – dodanie nowej funkcji (np. spania, łowienia ryb) wymaga jedynie stworzenia nowego stanu, bez ingerencji w istniejące.
- **Minimalizację błędów logicznych** – brak kolizji między warunkami odpowiedzialnymi za różne działania zmniejsza ryzyko konfliktów i nieprzewidzianych interakcji.

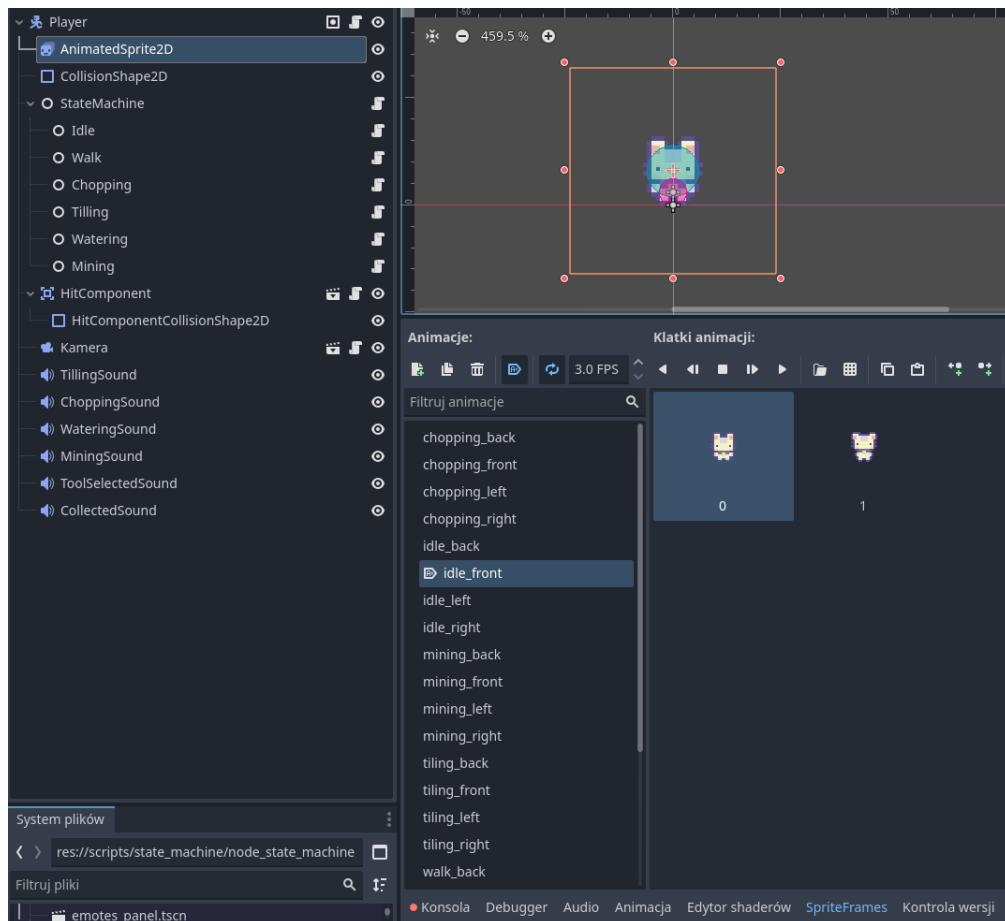
4.2.2 Architektura maszyny stanów

System składa się z następujących elementów:

- **NodeState** – klasa bazowa, definiująca interfejs stanu (metody cyklu życia),
- **NodeStateMachine** – zarządza aktualnym stanem, przełącza stany na podstawie sygnałów i deleguje wywołania metod,
- **Stany potomne** – takie jak **Idle**, **Walk**, **Chopping**, **Mining**, **Watering**, **Tilling**, każdy z własną logiką.

Każdy stan realizuje konkretną funkcję:

- odtwarza animację w zależności od kierunku postaci,
- ustawia pozycję aktywnego pola kolizji (gdzie narzędzie „uderza”),
- uruchamia dźwięk (np. `chopping_sound.play()`),
- przekazuje informacje do innych systemów (np. `TaskManager.report_progress()`),
- przełącza się automatycznie do innego stanu po zakończeniu działania.



Rysunek 9: Struktura węzłów gracza i animacje przypisane w **AnimatedSprite2D**. Każdy stan (np. **Walk**, **Chopping**) odtwarza odpowiednią animację zależnie od kierunku postaci.

4.2.3 Przykład przejścia między stanami

Z poziomu stanu `Idle`, jeśli gracz naciśnie przycisk działania i posiada odpowiednie narzędzie, następuje zmiana stanu:

```
1 if player.current_tool == DataTypes.Tools.MineStone and GameInputEvents.use_tool():
2     transition.emit("Mining")
```

Z kolei stan `Mining` odtwarza animację i dźwięk, a po jej zakończeniu automatycznie przechodzi do stanu `Idle`:

```
1 if !animated_sprite_2d.is_playing():
2     transition.emit("Idle")
```

4.2.4 Integracja z innymi systemami

Maszyna stanów współdziała z innymi komponentami gry:

- **System zadań** – stany raportują postęp działań (np. podlewania),
- **System kolizji** – każdy stan aktywuje `CollisionShape2D` w odpowiednim miejscu i czasie pozwalający na używanie narzędzia,
- **Audio** – każdy stan odtwarza przypisany dźwięk (np. ścinanie, kopanie),
- **Animacje** – sterowanie `AnimatedSprite2D` odbywa się zależnie od kierunku poruszania się gracza.

4.3 System kamery

System kamery został zaprojektowany w sposób minimalistyczny, ale skuteczny, zapewniając płynne i czytelne śledzenie ruchów gracza w przestrzeni 2D. Kamera jest oparta na klasie `Camera2D` i przymocowana bezpośrednio do węzła gracza, dzięki czemu automatycznie podąża za jego pozycją.

4.3.1 Zasada działania

Zamiast sztywnego przypięcia pozycji kamery do gracza, zastosowano interpolację liniową (`lerp`), co pozwala na płynne "doganianie" gracza z delikatnym opóźnieniem. Dzięki temu ruch kamery jest bardziej naturalny i mniej gwałtowny, szczególnie podczas szybkiego poruszania się:

```
1 global_position = lerp(global_position, target.global_position, follow_speed * delta)
2 global_position = global_position.round()
```

Kod 1: Interpolacja pozycji kamery

Parametr `follow_speed` określa tempo, z jakim kamera nadąża za graczem. Funkcja `round()` służy do dopasowania pozycji kamery do pełnych pikseli, co zapobiega rozmyciu obrazu.

4.3.2 Integracja z hierarchią sceny

Kamera znajduje się bezpośrednio jako dziecko obiektu gracza i automatycznie staje się aktywna w momencie uruchomienia gry:

```
1 func _ready():
2     target = get_parent() # czyli gracz
3     make_current()
```

Nie ma potrzeby ręcznego aktualizowania pozycji, ponieważ system opiera się na aktualnej pozycji gracza w każdym cyklu `_process()`.

4.3.3 Optymalizacja pod grafikę pikselową

Aby zachować ostrość i czytelność grafik w stylu pixel-art, w projekcie zastosowano odpowiednie ustawienia projektu:

- Project > Rendering > 2D > Snapping > Use Pixel Snap = ON
- Project > Display > Window > Stretch Mode = pixel
- Filter: OFF – wyłączone filtrowanie tekstur

Włączenie Use Pixel Snap powoduje, że wszystkie elementy 2D (sprite'y, kamera, UI) są renderowane w oparciu o siatkę pikseli, co eliminuje efekt rozmycia i zapewnia wyraźne kontury niezależnie od rozdzielczości ekranu.

4.3.4 Zalety zastosowanego rozwiązania

Prosty system kamery z interpolacją i dopasowaniem do siatki pikseli ma wiele zalet:

- **Płynność ruchu** – interpolacja sprawia, że przesuwanie obrazu jest miękkie i naturalne,
- **Estetyka grafiki** – brak rozmycia sprite'ów dzięki snapowaniu pikseli i filtrowaniu,
- **Niskie koszty obliczeniowe** – brak zbędnych obliczeń lub komponentów,
- **Brak widocznych skoków kamery** – nawet przy dynamicznych zmianach kierunku gracza.

Rozwiązanie to w pełni wystarcza na potrzeby gry 2D z widokiem z góry i zostało zaprojektowane z myślą o prostocie oraz kompatybilności z grafiką typu pixel-art.

4.4 System używania narzędzi

Używanie narzędzi w grze *Bloomrise* pozwala graczowi na interakcję z obiektami świata – głównie poprzez ich niszczenie (np. drzewa, skały) lub przekształcanie (np. przekopywanie ziemi, podlewanie roślin). Mechanizm opiera się na komunikacji pomiędzy komponentami: `HitComponent`, `HurtComponent`, `DamageComponent` oraz opcjonalnie `CollectableComponent`.

4.4.1 Komponenty i ich role

System używa modularnego modelu interakcji:

- **HitComponent** – znajduje się po stronie gracza. Definiuje aktualnie używane narzędzie oraz siłę uderzenia.
- **HurtComponent** – przypisany do obiektu interaktywnego. Weryfikuje poprawność narzędzia i emituje sygnał uszkodzenia.
- **DamageComponent** – gromadzi obrażenia i informuje o osiągnięciu maksymalnego poziomu zniszczenia.
- **CollectableComponent** – odpowiada za zbieranie przedmiotów do ekwipunku po wejściu w kolizję z graczem.

Komponent HitComponent

Znajduje się po stronie gracza i jest odpowiedzialny za przekazanie informacji o aktualnie używanym narzędziu oraz sile ataku:

```
1 class_name HitComponent
2 extends Area2D
3
4 @export var current_tool : DataTypes.Tools = DataTypes.Tools.None
5 @export var hit_damage : int = 1
```

Kod 2: HitComponent – konfiguracja narzędzia i siły

Komponent HurtComponent

Dodawany do obiektów, które mogą zostać zniszczone. Oczekuje uderzenia odpowiednim narzędziem i emituje sygnał `hurt`:

```
1 class_name HurtComponent
2 extends Area2D
3
4 @export var tool : DataTypes.Tools = DataTypes.Tools.None
5 signal hurt
6
7 func _on_area_entered(area: Area2D) -> void:
8     var hit_component = area as HitComponent
9     if tool == hit_component.current_tool:
10         hurt.emit(hit_component.hit_damage)
```

Kod 3: HurtComponent – weryfikacja narzędzia

Komponent DamageComponent

Obsługuje aktualny stan uszkodzenia obiektu i emituje sygnał po osiągnięciu maksymalnych obrażeń:

```
1 class_name DamageComponent
2 extends Node2D
3
4 @export var max_damage = 1
5 @export var current_damage = 0
6
7 signal max_damaged_reached
8
9 func apply_damage(damage: int) -> void:
10     current_damage = clamp(current_damage + damage, 0, max_damage)
11     if current_damage == max_damage:
12         max_damaged_reached.emit()
```

Kod 4: DamageComponent – akumulacja obrażeń

Po odebraniu sygnału `max_damaged_reached`, obiekt może utworzyć inny zasób (np. fragment skały) i zniknąć ze sceny:

```
1 func on_max_damage_reached() -> void:
2     call_deferred("add_stone_scene")
3     queue_free()
4
5 func add_stone_scene() -> void:
6     var stone_instance = stone_scene.instantiate()
7     stone_instance.global_position = global_position
8     get_parent().add_child(stone_instance)
```

Kod 5: Zachowanie po zniszczeniu obiektu

Komponent CollectableComponent

Odpowiada za umożliwienie zebrania przedmiotu do ekwipunku. Po wykryciu ciała gracza wywołuje metodę dodającą przedmiot do zasobu `InventoryManager`:

```
1 class_name CollectableComponent
2 extends Area2D
3
4 @export var collectable_name: String
5
6 func _on_body_entered(body: Node2D) -> void:
7     if body is Player:
8         var was_collected = InventoryManager.add_collectable(collectable_name)
9         if was_collected:
10             SceneManager.play_collect_sound()
11             get_parent().queue_free()
```

Kod 6: CollectableComponent – zbieranie przedmiotów

4.4.2 Struktura obiektów interaktywnych

System narzędzi wspiera dwa główne typy obiektów interaktywnych:

Obiekty niszczone (np. `BigStone.tscn`)

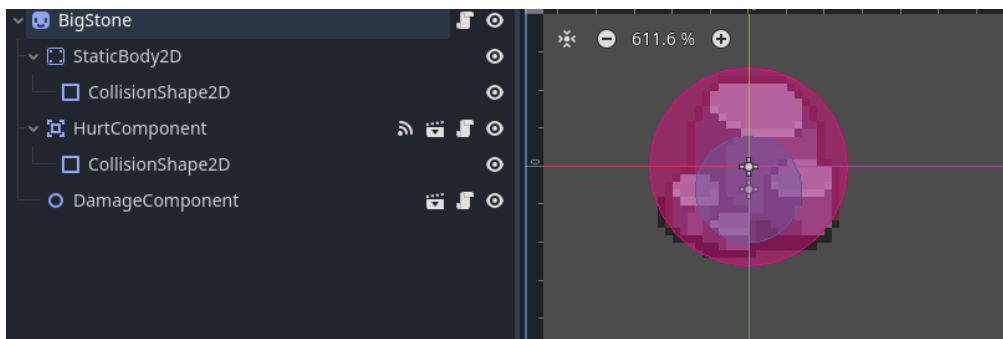
Zawierają komponenty:

- `HurtComponent` – wykrywa uderzenie narzędziem i sprawdza jego typ,
- `DamageComponent` – zarządza progiem uszkodzenia,
- `StaticBody2D` – zapewnia obecność fizyczną w świecie.

Po zniszczeniu uruchamiana jest logika podmiany na zasób zbieralny:

```
1 func on_max_damage_reached() -> void:  
2     call_deferred("add_stone_scene")  
3     queue_free()  
4  
5 func add_stone_scene() -> void:  
6     var stone_instance = stone_scene.instantiate()  
7     stone_instance.global_position = global_position  
8     get_parent().add_child(stone_instance)
```

Kod 7: Usunięcie i podmiana obiektu

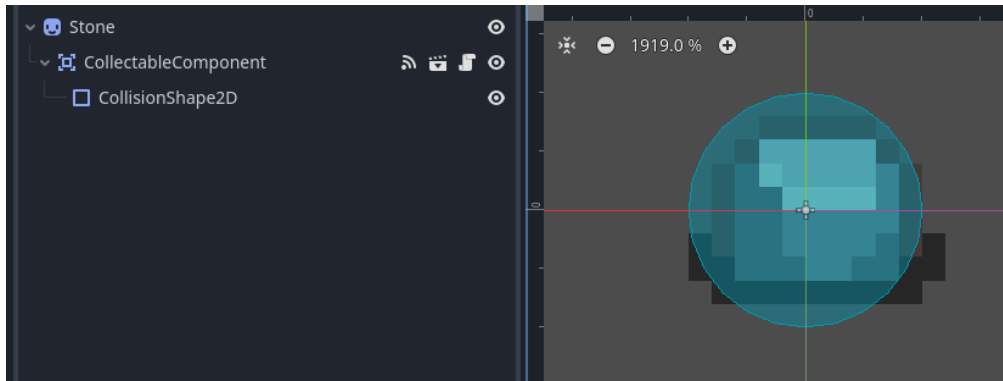


Rysunek 10: Struktura obiektu niszczonego `BigStone.tscn`.

Obiekty zbieralne (np. `Stone.tscn`)

Tworzone po zniszczeniu większych struktur. Zawierają:

- `CollectableComponent` – obsługuje logikę zebrania,
- `CollisionShape2D` – definiuje strefę wykrywania gracza.



Rysunek 11: Struktura obiektu zbieralnego `Stone.tscn`.

4.4.3 Mechanika działania

Cykl interakcji przebiega według następujących kroków:

1. Gracz aktywuje narzędzie (konfiguracja `HitComponent`).
2. Obiekt (np. drzewo) z `HurtComponent` wykrywa kolizję i sprawdza poprawność narzędzia.
3. Jeśli wybrano odpowiednie narzędzie – emitowany jest sygnał `hurt`, który trafia do `DamageComponent`.
4. Po osiągnięciu progu zniszczenia, obiekt zostaje usunięty, a jego miejsce zajmuje obiekt zbieralny.
5. Obiekt z `CollectableComponent` może zostać podniesiony przez gracza.

4.4.4 Zalety systemu

Modularność systemu i utworzenie dodatkowych komponentów pozwala na:

- proste przypisanie narzędzi do konkretnych obiektów,
- wielokrotne użycie komponentów bez ich duplikacji,
- łatwą rozbudowę o nowe interakcje
- utrzymanie przejrzystości kodu i scen.

4.5 System czasu

System dobowy zarządza upływem czasu w grze i jest implementowany w klasie `DayAndNightCycleManager`. Używa wewnętrznej zmiennej `time`, która rośnie proporcjonalnie do czasu rzeczywistego (`delta`) i prędkości gry (`game_speed`).

Cechy systemu:

- Wirtualny dzień trwa 24 godziny (1440 minut),
- Emitowane są sygnały `time_tick` (co minutę) i `time_tick_day` (co nowy dzień),
- Umożliwia łatwe dodawanie zachowań zależnych od pory dnia (np. wzrost roślin, zmiany pogody).

```

1 func recalculate_time() -> void:
2     var total_minutes: int = int(time / GAMES_MINUTE_DURATION)
3     var day: int = int(total_minutes / MINUTES_PER_DAY)
4     var current_day_minutes: int = total_minutes % MINUTES_PER_DAY
5     var hour: int = int(current_day_minutes / MINUTES_PER_HOUR)
6     var minute: int = int(current_day_minutes % MINUTES_PER_HOUR)
7

```

```

8  if current_minute != minute:
9      current_minute = minute
10     time_tick
11     time_tick.emit(day, hour, minute)
12
13  if current_day != day:
14      current_day = day
15      time_tick_day.emit(day)

```

Kod 8: Zmiana czasu

4.6 System pogody

System pogody w grze odpowiada za codzienne losowanie warunków atmosferycznych oraz informowanie innych systemów o ich zmianie. Jest zaimplementowany jako autonomiczny węzeł (WeatherSystem.gd) i działa w ścisłej współpracy z systemem czasu (DayAndNightCycleManager), reagując na sygnał nowego dnia.

Mechanizm działania:

- Pogoda reprezentowana jest jako enum:

```
enum Weather { SUNNY, CLOUDY, RAINY }
```

- Dzięki temu system jest czytelny i łatwy do rozbudowy o kolejne typy pogody (np. burza, śnieg).
- Aktualna pogoda jest przechowywana w zmiennej `current_weather` (domyślnie `SUNNY`).
- Za losowanie pogody odpowiada generator liczb losowych (`RandomNumberGenerator`), inicjalizowany i randomizowany w `_ready()`.
- Każdego nowego dnia wykonywana jest metoda `_on_new_day()`, która losuje pogodę zgodnie z ustalonymi prawdopodobieństwami:
 - 60% szans na słoneczną pogodę (`SUNNY`),
 - 20% szans na deszcz (`RAINY`),
 - 20% szans na zachmurzenie (`CLOUDY`).
- Po ustaleniu nowej pogody, system emituje sygnał `weather_changed(new_weather)`, który może być odbierany przez inne komponenty gry (np. rośliny, interfejs, efekty dźwiękowe i graficzne).

```

1  func _on_new_day(day: int) -> void:
2      var roll = rng.randf()
3      if roll < 0.6:
4          current_weather = Weather.SUNNY
5      elif roll < 0.8:
6          current_weather = Weather.RAINY
7      else:
8          current_weather = Weather.CLOUDY

```

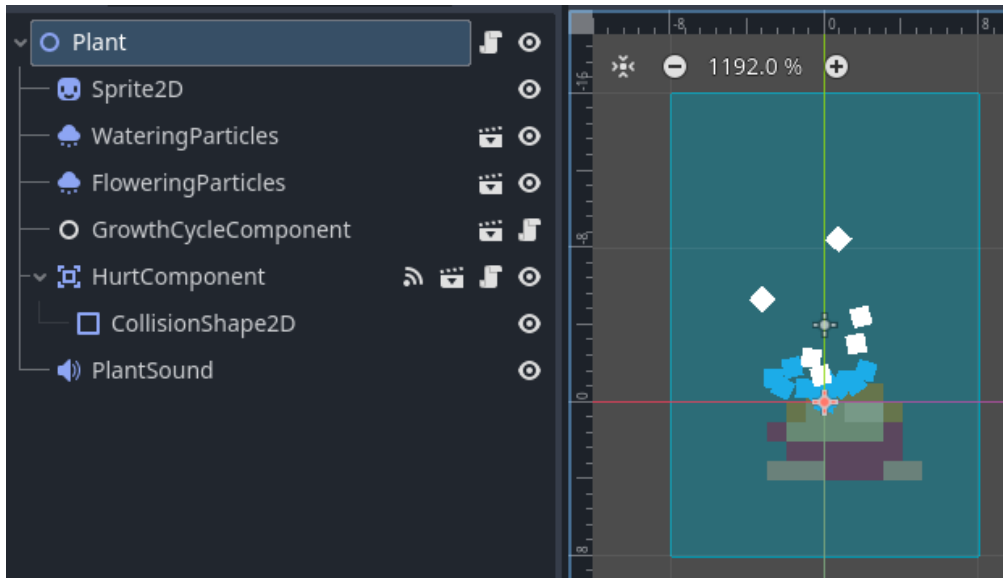
Kod 9: Losowanie pogody

4.7 System wzrostu roślin

System wzrostu roślin odpowiada za zarządzanie cyklem życia każdej zasianej rośliny w grze – od momentu posadzenia, przez kolejne etapy rozwoju, aż po dojrzałość i możliwość zbioru. Jest on zaprojektowany jako zestaw współpracujących komponentów, które reagują na zmiany czasu, pogody oraz interakcje gracza (np. podlewanie).

4.7.0.1 Struktura sceny

Scena rośliny składa się z obrazka rośliny załadowanego z atlasu roślin. Zawiera również `WateringParticles` wywoływane podczas podlewania oraz `FloweringParticles` wywoływane podczas kwitnienia. Oprócz tego został również dodany dźwięk wywoływany podczas sadzenia rośliny.



Rysunek 12: Widok sceny rośliny

Główna logika wzrostu rośliny została zaimplementowana w komponencie `GrowthCycleComponent`, który kontroluje aktualny etap rozwoju oraz warunki niezbędne do przejścia do kolejnej fazy. Komponent działa w oparciu o liczbę dni, które muszą minąć od początku wzrostu oraz o informacje pogodowe (np. deszcz podlewa automatycznie).

```
1 class_name GrowthCycleComponent
2 extends Node
3
4 @export var current_growth_state: DataTypes.GrowthStates = DataTypes.GrowthStates.
   Germination
5 @export_range(5, 365) var days_until_harvest: int = 8
6
7 signal crop_maturity
8 signal crop_harvesting
9 signal growth_state_changed(new_state: int)
```

Kod 10: Struktura `GrowthCycleComponent`

4.7.0.2 Cykl życia rośliny

Roślina przechodzi przez cztery główne stany rozwoju zdefiniowane w `DataTypes.GrowthStates`: kiełkowanie, wyrastanie, kwitnienie oraz dojrzałość. Każdy stan trwa przez określoną liczbę dni (parametr `days_per_state`), a przejście do kolejnego wymaga, by roślina została podlana.

Dla każdej rośliny liczony jest `starting_day`, który reprezentuje dzień posadzenia. Na podstawie tego dnia oraz informacji o podlewaniu system decyduje, czy można przejść do następnego etapu.

4.7.0.3 Logika przejścia między stanami

Najważniejsza funkcja `growth_states` sprawdza, czy warunki do rozwoju są spełnione i zmienia stan rozwoju, jeśli jest to możliwe.

```

1 func growth_states(starting_day: int, current_day: int) -> void:
2     if current_growth_state == DataTypes.GrowthStates.Maturity or current_growth_state ==
        DataTypes.GrowthStates.Harvesting:
3         harvest_state(starting_day, current_day)
4         return
5
6     var new_state = clamp(current_growth_state + 1, 0, DataTypes.GrowthStates.Maturity)
7
8     if current_growth_state != new_state:
9         grow_day = current_day
10        current_growth_state = new_state
11        growth_state_changed.emit(current_growth_state)
12
13        if current_growth_state == DataTypes.GrowthStates.Maturity:
14            crop_maturity.emit()

```

Kod 11: Zmiana stanu wzrostu

4.7.0.4 Podlewanie i wpływ pogody

Każda roślina wymaga podlewania, aby mogła rosnąć. Jeśli nie została podlana w danym cyklu (dni między stanami), nie przejdzie do kolejnej fazy. Dodatkowo, w przypadku deszczowej pogody system automatycznie oznacza roślinę jako podlaną:

```

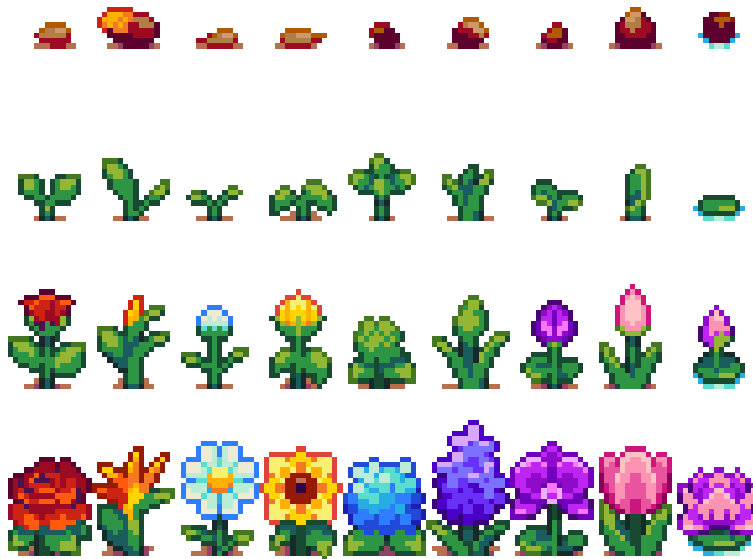
1 func on_weather_changed(new_weather: int) -> void:
2     if new_weather == WeatherManager.Weather.RAINY:
3         is_watered_today()
4
5 func is_watered_today():
6     watered_on_day = DayAndNightCycleManager.current_day

```

Kod 12: Automatyczne podlewanie przy deszczu

4.7.0.5 Wizualizacja etapów wzrostu

Wizualna reprezentacja rośliny zmienia się na podstawie bieżącego etapu wzrostu. Sprity są odczytywane z atlasu graficznego, w którym kolejne etapy wzrostu są rozmieszczone pionowo. Dodatkowo, efekt cząsteczek (particles) zaczyna być aktywowany po osiągnięciu dojrzałości.



Rysunek 13: Atlas roślin

```
1 func update_sprite_for_state(state: int) -> void:
2     sprite_2d.frame = frames_per_stage * state + start_frame_offset
```

Kod 13: Aktualizacja sprite'u rośliny

4.7.0.6 Zbiór i zakończenie cyklu

Po osiągnięciu ostatniego stanu ('Harvesting'), roślina może zostać zebrana. Podczas zbioru tworzony jest nowy obiekt (kwiat rośliny), a sama roślina zostaje usunięta z gry:

```
1 func on_crop_harvesting() -> void:
2     var harvest_instance = harvest_scene.instantiate() as Node2D
3     harvest_instance.global_position = global_position
4     get_parent().add_child(harvest_instance)
5     queue_free()
```

Kod 14: Zbiór dojrzałej rośliny

4.7.0.7 Integracja z innymi systemami

System wzrostu roślin ściśle współpracuje z menedżerem czasu (`DayAndNightCycleManager`) oraz systemem pogody (`WeatherManager`). Wydarzenia takie jak zmiana dnia lub opady deszczu wywołują odpowiednie reakcje w komponencie rośliny. Dodatkowo, interakcja gracza (np. podlewanie) wpływa na postęp wzrostu.

4.7.0.8 Rozszerzalność

Obecna architektura umożliwia łatwe dodanie nowych rodzajów roślin (poprzez `start_frame_offset`) oraz rozbudowę o dodatkowe fazy wzrostu, efekty, czy unikalne wymagania (np. światło, gleba). Dzięki temu system może zostać rozszerzony w przyszłości o bardziej złożoną mechanikę ogrodniczą.

4.8 System sadzenia roślin

System sadzenia roślin w grze składa się z dwóch głównych komponentów:

- `FieldCursorComponent` – odpowiada za tworzenie i usuwanie przekopanej gleby (`tilled soil`),
- `CropsCursorComponent` – odpowiada za tworzenie i usuwanie obiektów roślin (`plant.tscn`) na przygotowanej glebie.

1. Tworzenie gleby pod uprawy (`FieldCursorComponent`)

- Po wybraniu narzędzia `TillGround` i użyciu przycisku akcji (`hit`):

1. Obliczana jest pozycja uderzenia motyki na podstawie kierunku i pozycji gracza.

```
1 func get_tiller_hit_position(player_position: Vector2, player_direction: Vector2)
  -> Vector2:
2   var offset := Vector2.ZERO
3   print(player_direction)
4
5   match player_direction:
6     Vector2.UP:
7       offset = Vector2(0, -18)
8     Vector2.RIGHT:
9       offset = Vector2(9, 0)
10    Vector2.LEFT:
11      offset = Vector2(-9, 0)
12    _:
13      offset = Vector2(0, 3)
14
15   return player_position + offset + Vector2(12, 12)
```

2. Wyznaczana jest odpowiednia komórka w warstwie `grass_tilemap_layer`.

```
1 func get_cell_under_tiller() -> void:
2   var player_direction = player.get_player_direction()
3   var player_global_position = player.global_position
4   var tiller_global_position = get_tiller_hit_position(player_global_position,
5     player_direction)
6   var tiller_local_position = grass_tilemap_layer.to_local(tiller_global_position)
7   cell_position = grass_tilemap_layer.local_to_map(tiller_local_position)
8   local_cell_position = grass_tilemap_layer.map_to_local(cell_position)
```

3. Na odpowiadającej pozycji w `tilled_soil_tilemap_layer` ustawiana jest gleba z terenem `terrain_set` i `terrain` (ziemia uprawna).

```
1 func add_tilled_dirt_cell() -> void:
2   if cell_source_id != -1:
3     tiled_soil_tilemap_layer.set_cells_terrain_connect(
4       [cell_position], terrain_set, terrain, true)
```

- Użycie przycisku `unhit` z tym samym narzędziem usuwa wcześniej dodany kafel gleby.

2. Sadzenie i usuwanie roślin (`CropsCursorComponent`)

- Gracz wybiera jedno z narzędzi sadzenia nasion (np. `RedRoseSeeds`).
- Użycie przycisku akcji (`hit`):
 1. Obliczana jest najbliższa komórka `tilled_soil_tilemap_layer` względem gracza.
 2. Sprawdzane jest, czy w promieniu 20 jednostek od gracza znajduje się dana komórka.
 3. Jeśli komórka jest pusta, instancjonowany jest prefabrykat sceny `plant.tscn`.

4. Do instancji rośliny przypisywany jest offset odpowiadający wybranemu nasionu (ramka startowa animacji rośliny).
5. Roślina jest dodawana do kontenera `CropFileds`.

```

1 if is_cell_empty(local_cell_position):
2     var plant_instance = plant_scene.instantiate() as Node2D
3     var offset = tool_to_flower_offset.get(ToolManager.selected_tool, 0)
4     plant_instance.set("start_frame_offset", offset)
5     plant_instance.global_position = tilled_soil_tilemap_layer.to_global(
6         local_cell_position)
7     crop_fileds.add_child(plant_instance)

```

- Użycie przycisku `unhit` usuwa najbliższą roślinę w promieniu 5 jednostek od komórki.

3. Mapa narzędzi do typów roślin

Dzięki słownikowi `tool_to_flower_offset`, każde nasiona odpowiada konkretnej klatce animacji (offsetowi):

- `DataTypes.Tools.RedRoseSeeds` → klatka 0
- `DataTypes.Tools.OrangeLilySeeds` → klatka 1
- itd.

4. Zależności komponentów

- Komponent `CropsCursorComponent` wymaga aktywnej warstwy `tilled_soil_tilemap_layer` oraz obecności kontenera `CropFileds` jako rodzica.
- Komponent `FieldCursorComponent` działa w kontekście `grass_tilemap_layer`, w której orientuje się w świecie.
- Oba komponenty bazują na aktualnej pozycji gracza, pozyskiwanej dynamicznie.

4.9 System ekwipunku, paska narzędzi i skrzyń

Gra *Bloomrise* oferuje pełnoprawny system zarządzania zasobami, który obejmuje:

- **Ekwipunek gracza (Inventory)** – zbiór przedmiotów zdobytych w grze, przechowywanych w siatce 4x3 slotów.
- **Pasek narzędzi (Toolbar)** – osobny panel dostępny podczas rozgrywki, umożliwiający szybki dostęp do aktywnych narzędzi.
- **Skrzynie (Chest)** – zewnętrzne magazyny do długoterminowego przechowywania przedmiotów.

System został zaprojektowany tak, aby wszystkie elementy korzystały z jednolitych zasobów danych:

- `InvItem` – zasób reprezentujący pojedynczy przedmiot (nazwa, tekstura, cena sprzedaży),
- `InvSlot` – struktura przechowująca instancję przedmiotu oraz jego ilość,
- `update_slots` – sygnał emitowany przez zasoby `Inventory` i `Chest`, umożliwiający synchronizację z UI.

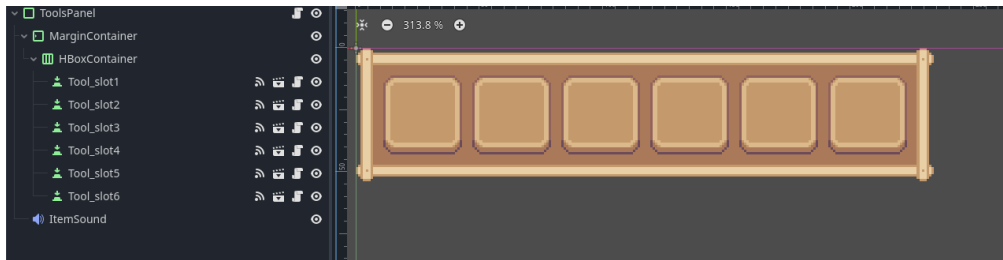
Dzięki temu, przenoszenie przedmiotów między ekwipunkiem, paskiem i skrzynią odbywa się na tych samych zasadach i może być zrealizowane za pomocą kliknięcia lub kombinacji klawiszy.

4.9.1 Pasek narzędzi (ToolsPanel)

Pasek narzędzi jest panelem z sześcioma dedykowanymi slotami, zrealizowanymi jako przyciski typu `tool_slot.gd`. Każdy slot zawiera:

- `ToolIcon` (`TextureRect`) – przedstawia ikonę narzędzia,
- `CenterContainer` – centruje grafikę w obrębie slotu,
- zmienne przechowujące typ narzędzia i odniesienie do zasobu `InvItem`.

Węzły te znajdują się w scenie `ToolsPanel`, osadzonej na stałe w interfejsie użytkownika, widocznej w czasie rozgrywki.



Rysunek 14: Struktura sceny paska narzędzi (`ToolsPanel`) z sześcioma slotami (`Tool_slot1`–`Tool_slot6`), umieszczonymi w `HBoxContainer`. Każdy slot służy do przechowywania i aktywacji jednego narzędzia.

4.9.1.1 Dodawanie narzędzi

Do paska można dodać narzędzie np. poprzez interakcję z ekwipunkiem (LPM), samouczek lub skrypt systemowy. Obsługa opiera się o funkcję:

```
1 func add_item_to_first_free_slot(item: InvItem) -> bool:
2     for slot in tool_slots:
3         if slot.is_empty():
4             slot.set_item(item)
5             return true
6     return false
```

Kod 15: Dodanie narzędzia do pierwszego wolnego slotu na pasku

Funkcja ta iteruje po wszystkich sześciu slotach na pasku narzędzi (`Tool_slot1`–`Tool_slot6`), próbując znaleźć pierwszy pusty. Jeśli się uda, wywołuje metodę `set_item(item)`, która przypisuje dane narzędzie do slotu.

Wewnątrz `set_item()` sprawdzane jest, czy przedmiot faktycznie należy do typu narzędzi. W tym celu wykorzystywane jest mapowanie `tool_name_to_type`, które zawiera listę rozpoznawanych nazw:

```
1 if tool_name_to_type.has(item.name):
2     tool_data = tool_name_to_type[item.name]
3     tool_icon.texture = item.texture
4     tool_icon.visible = true
```

Kod 16: Walidacja typu narzędzia przed dodaniem

Dzięki temu nie można dodać do paska np. zwykłych przedmiotów, które nie mają przypisanego działania (jak drewno czy kamień).

4.9.1.2 Logika i interakcja ze slotem narzędzi

Każdy slot narzędziowy rozszerza klasę `Button` i pełni jednocześnie funkcję kontenera na narzędzie oraz przycisku, który reaguje na interakcje użytkownika. Skrypt `tool_slot.gd` obsługuje:

- przypisanie narzędzia do slotu za pomocą metody `set_item()`,
- walidację, czy dany przedmiot jest rozpoznawanym narzędziem (poprzez mapę `tool_name_to_type`),
- aktywację narzędzia po kliknięciu LPM,
- przenoszenie narzędzia z powrotem do ekwipunku po kliknięciu PPM,
- synchronizację z systemem gracza (`ToolManager`) przez emisję sygnału `tool_selected`.

```

1 func set_item(item):
2     if tool_name_to_type.has(item.name):
3         tool_data = tool_name_to_type[item.name]
4         item_ref = item
5         tool_icon.texture = item.texture
6         tool_icon.visible = true
7
8 func _on_pressed() -> void:
9     if not is_empty():
10        ToolManager.select_tool(tool_data)

```

Kod 17: Przypisanie narzędzia do slotu i jego aktywacja

```

1 func _gui_input(event: InputEvent) -> void:
2     if event.button_index == MOUSE_BUTTON_RIGHT and event.pressed:
3         if not is_empty():
4             var success = InventoryManager.inventory.insert(item_ref)
5             if success:
6                 clear_slot()

```

Kod 18: Zwracanie narzędzia do ekwipunku po kliknięciu PPM

Po aktywacji narzędzia (LMB), sygnał `tool_selected` jest przechwytywany przez obiekt gracza, co pozwala ustawić aktualne narzędzie i powiązać je z komponentem kolizji:

```

1 ToolManager.tool_selected.connect(on_tool_selected)
2
3 func on_tool_selected(tool: DataTypes.Tools):
4     current_tool = tool
5     hit_component.current_tool = tool

```

Kod 19: Reakcja gracza na wybór narzędzia

Dzięki temu system maszyny stanów może reagować na wybór narzędzia, np. przechodząc do stanu `Mining`, `Watering` lub `Chopping`.



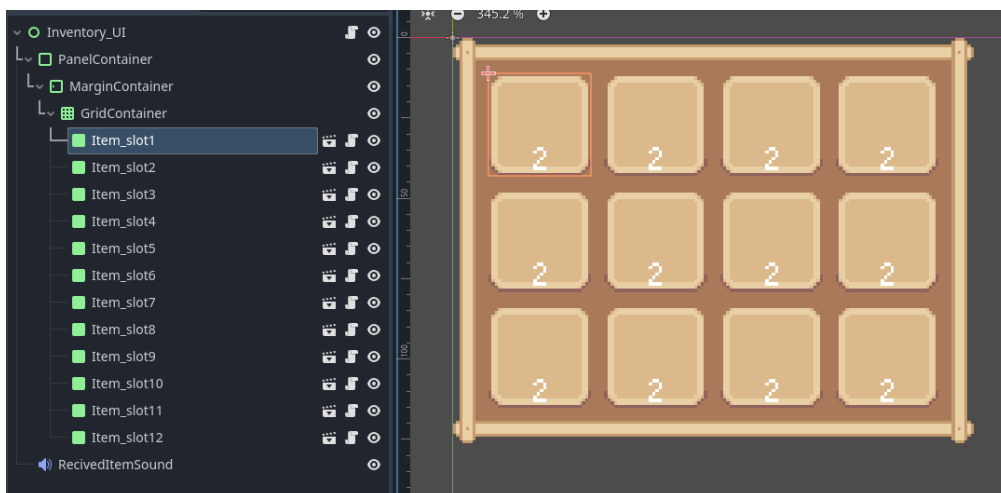
Rysunek 15: Widok gry z aktywnym paskiem narzędzi. Gracz wybrał kilof (pierwszy slot), który został wyróżniony wizualnie. Aktywne narzędzie jest używane przez system maszyny stanów do uruchomienia odpowiedniej animacji i logiki (kopanie kamieni).

4.9.2 Ekwipunek gracza

Ekwipunek to podstawowy system przechowywania przedmiotów, do którego trafiają wszystkie zdobyte przez gracza surowce i obiekty użytkowe. Gracz może zarządzać zawartością ekwipunku w dedykowanym panelu `Inventory_UI`, który domyślnie jest ukryty i otwierany za pomocą klawisza przypisanego do akcji `inventory_open` (E) .

4.9.2.1 Struktura sceny

Scena `Inventory_UI` zawiera dwanaście slotów (obiekty `Item_slot`), rozmieszczonych w siatce `GridContainer`. Każdy slot umożliwia wyświetlenie ikony przedmiotu, jego liczby oraz obsługę interakcji myszą. System korzysta z danych typu `InvSlot`, które przechowują odniesienie do przedmiotu (`InvItem`) oraz jego ilość.



Rysunek 16: Struktura sceny `Inventory_UI` – siatka dwunastu slotów ekwipunku.

4.9.2.2 Dodawanie przedmiotów

Nowe przedmioty trafiają do ekwipunku za pomocą metody `insert()` w skrypcie `inventory.gd`. Jeśli przedmiot już znajduje się w jednym ze slotów, jego ilość jest zwiększana. W przeciwnym razie gra próbuje znaleźć pusty slot. Jeśli ekwipunek jest pełny (wszystkie 12 slotów zawiera przedmiot), metoda zwraca `false` i przedmiot nie zostaje dodany.

```
1 func insert(item: InvItem) -> bool:
2     var itemslots = slots.filter(func(slot): return slot.item == item)
3     if !itemslots.is_empty():
4         itemslots[0].amount += 1
5         update_slots.emit()
6         return true
7
8     var emptyslots = slots.filter(func(slot): return slot.item == null)
9     if !emptyslots.is_empty():
10        emptyslots[0].item = item
11        emptyslots[0].amount = 1
12        update_slots.emit()
13        return true
14
15    print("Inventory full: max 12 item types.")
16    return false
```

Kod 20: Logika dodawania przedmiotów do ekwipunku

4.9.2.3 Logika pojedynczego slotu – `item_slot.gd`

Każdy slot to niezależny komponent odpowiedzialny za wizualizację przedmiotu i jego obsługę. Klasa `item_slot.gd` rozszerza `Panel` i zawiera:

- ikonę przedmiotu (`item_icon`),
- etykietę liczby sztuk (`amount_label`),
- dane slotu (`slot_data: InvSlot`),
- referencję do otwartej skrzyni (`parent_chest`), jeśli dotyczy.

Przy kliknięciu:

- PPM – wyrzuca przedmiot na ziemię,
- LPM – przenosi przedmiot do paska narzędzi,
- Shift + LPM – transfer między ekwipunkiem a skrzynią.

```
1 if event.button_index == MOUSE_BUTTON_RIGHT:
2     ItemDropManager.spawn_drop(slot_data.item, ...)
3     slot_data.amount -= 1
4
5 elif event.button_index == MOUSE_BUTTON_LEFT:
6     if Input.is_key_pressed(KEY_SHIFT):
7         transfer_to_or_from_chest()
8     else:
9         emit_signal("item_sent_to_toolbar", slot_data)
```

Kod 21: Przykład obsługi kliknięcia w slot

Slot dba o synchronizację z danymi poprzez ręczne aktualizowanie widoku (`update()`) oraz emisję odpowiednich sygnałów. Obsługuje też rozróżnienie kontekstu (czy znajduje się w skrzyni, czy w ekwipunku gracza).

4.9.2.4 Synchronizacja danych

Widok ekwipunku jest zsynchronizowany z danymi zasobu `Inv`, a zmiany są przekazywane przez sygnał `update_slots`. Dzięki temu każda zmiana zawartości (np. dodanie, usunięcie, przeniesienie) natychmiast odświeża interfejs i aktualizuje sloty w UI.

4.9.2.5 Wyrzucanie przedmiotów

Gracz może wyrzucać przedmioty z ekwipunku poprzez kliknięcie **prawego przycisku myszy (RMB)** na slotcie przedmiotu. Wówczas system dynamicznie tworzy w świecie gry nowy obiekt reprezentujący fizyczny zasób, który można później zebrać – np. kamień, kwiat, sadzonkę.

Za działanie tej funkcjonalności odpowiada singleton `ItemDropManager`. Posiada on metodę `spawn_drop(item, pos)`, której zadaniem jest:

1. sprawdzić, czy dany przedmiot ma przypisaną gotową scenę do zrzutu (`InventoryManager.drop_scene_db[item.name]`)
2. zainstancjonować scenę obiektu (np. `stone.tscn`),
3. umieścić ją w świecie gry w określonej pozycji,
4. dodać ją jako dziecko bieżącej sceny.

```
1 func spawn_drop(item: InvItem, pos: Vector2):
2     if not InventoryManager.drop_scene_db.has(item.name):
3         push_warning("No drop scene for item: " + item.name)
4         return
5
6     var drop_scene = InventoryManager.drop_scene_db[item.name]
7     var drop = drop_scene.instantiate()
8
9     drop.global_position = pos
10    get_tree().current_scene.add_child(drop)
```

Kod 22: Logika spawnowania wyrzuconego przedmiotu – `ItemDropManager.gd`

Nowo utworzony obiekt zazwyczaj zawiera `CollectableComponent`, co oznacza, że po podejściu gracza może zostać automatycznie podniesiony i wrócić do ekwipunku. Przykładowo: gracz może wyrzucić sadzonkę, a potem zebrać ją ponownie.

4.9.2.6 Efekty dźwiękowe i zamykanie

Ekwipunek posiada dźwięk odbierania przedmiotu (`RecivedItemSound`), odtwarzany przy przenoszeniu przedmiotu z ekwipunku na pasek. UI można zamykać i otwierać dynamicznie w trakcie gry bez przerywania rozgrywki.

4.9.3 Skrzynie i przechowywanie przedmiotów

Skrzynie w grze *Bloomrise* działają jako rozszerzenie systemu ekwipunku – oferują graczowi dodatkową przestrzeń do przechowywania przedmiotów, szczególnie przydatną, gdy jego własny inwentarz się zapełni.

4.9.3.1 Struktura i interakcja

Każda skrzynia w grze składa się ze sceny `Chest`, która zawiera animację otwierania, komponenty kolizyjne oraz skrypt `chest.gd` zarządzający jej logiką. Obiekt jest interaktywny – gracz może go otworzyć, jeśli znajduje się w jego zasięgu (zdefiniowanym przez `CollisionShape2D` oraz komponent `InteractableComponent`).

Strefa wykrywania oparta na kolizji umożliwia grze rozpoznanie, czy gracz znajduje się wystarczająco blisko. Po wejściu w zasięg interakcji wyświetlany jest dynamiczny wskaźnik z literą „R” (`InteractableLabelComponent`), informujący gracza o dostępnej akcji. To element systemu interakcji kontekstowej, który zwiększa czytelność interfejsu bez konieczności stałych komunikatów na ekranie.

Otwarcie skrzyni wywołuje metodę `toggle()`, która:

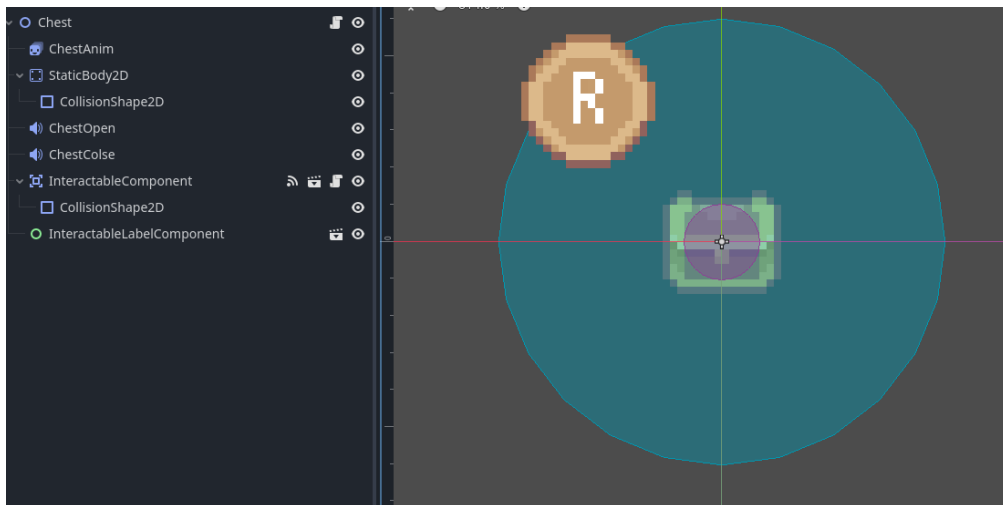
- odtwarza animację otwierania lub zamykania,
- aktywuje lub ukrywa interfejs ChestUI,
- sprawdza dystans od gracza, by zamknąć skrzynię gdy ten odejdzie od niej zbyt daleko,
- wyświetla lub ukrywa ikonę interakcji.

```

1 if in_range and Input.is_action_just_pressed("show_dialogue"):
2     toggle()
3
4 func toggle():
5     var distance = position.distance_to(player.position)
6     if not is_open and distance > 50:
7         return
8     if is_open:
9         chest_anim.play("chest_close")
10        chest_colse_sound.play()
11        chest_ui.close()
12    else:
13        chest_anim.play("chest_open")
14        chest_open_sound.play()
15        chest_ui.open_for(chest_data)

```

Kod 23: Fragment logiki otwierania skrzyni



Rysunek 17: Struktura sceny skrzyni w Godot — widoczne komponenty odpowiedzialne za kolizję, animację oraz obszar interakcji. Niebieskie pole to strefa wykrywania gracza, a ikona „R” sugeruje możliwość otwarcia skrzyni.

4.9.3.2 Reprezentacja danych skrzyni

Każda skrzynia posiada powiązany zasób typu `Chest`, który dziedziczy po `Resource` i przechowuje dwa nacie slotów (`InvSlot`). Operacje na skrzyni (dodawanie, aktualizacja) bazują na metodzie `insert()`:

```

1 func insert(item: InvItem) -> bool:
2     var itemslots = slots.filter(func(slot): return slot.item == item)
3
4     if !itemslots.is_empty():
5         itemslots[0].amount += 1
6         update_slots.emit()
7         return true
8
9     var emptyslots = slots.filter(func(slot): return slot.item == null)

```

```

10     if !emptyslots.is_empty():
11         emptyslots[0].item = item
12         emptyslots[0].amount = 1
13         update_slots.emit()
14         return true
15
16     print("Skrzynia pełna: max 12 typów przedmiotów.")
17     return false

```

Kod 24: Dodawanie przedmiotów do skrzyni

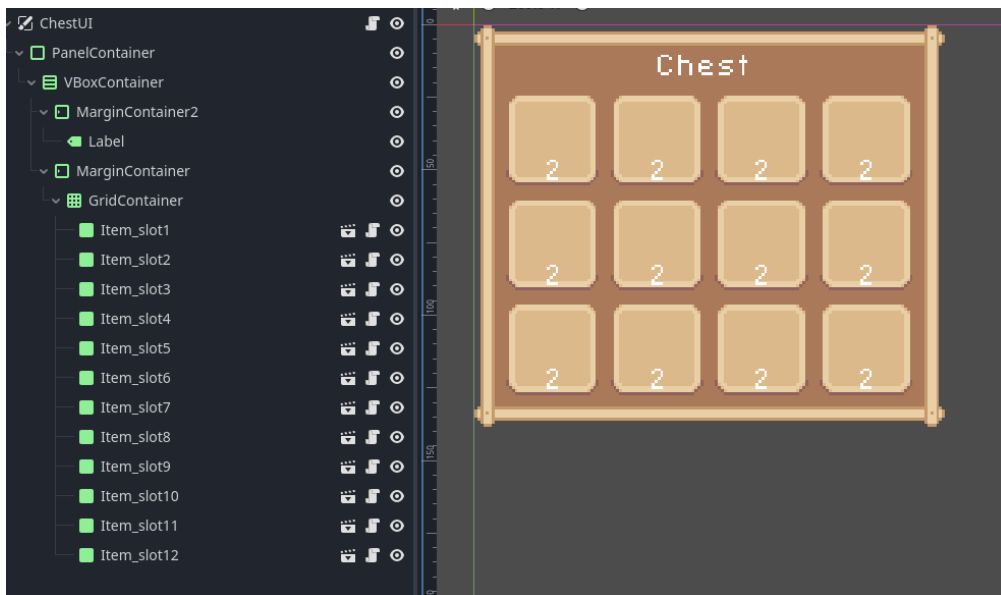
4.9.3.3 UI skrzyni

Interfejs użytkownika skrzyni (ChestUI) jest bardzo zbliżony do ekwipunku gracza – również wykorzystuje dwanaście slotów typu `Item_slot`, ułożonych w siatkę. Po otwarciu skrzyni funkcja `open_for(chest)` przypisuje aktywną skrzynię i odświeża zawartość:

```

1 func open_for(chest_ref: Chest):
2     chest = chest_ref
3     chest.update_slots.connect(update_slots)
4     update_slots()
5     visible = true

```



Rysunek 18: Interfejs użytkownika skrzyni. Widoczna zawartość i siatka slotów.

4.9.3.4 Przenoszenie przedmiotów

Gracz może przenosić przedmioty między skrzynią a ekwipunkiem za pomocą skrótu **Shift + LPM** na slotcie. W zależności od kontekstu:

- jeśli kliknięto w slot ekwipunku – przedmiot trafia do skrzyni,
- jeśli kliknięto w slot skrzyni – przedmiot trafia do ekwipunku.

Mechanizm ten jest zaimplementowany bezpośrednio w `item_slot.gd`, który sprawdza, czy slot pochodzi z inwentarza czy ze skrzyni i wykonuje odpowiedni transfer:

```

1 if parent_chest == null:
2     var chest_ui = get_tree().get_current_scene().get_node("YSorting/Chest/ChestUI")
3     if chest_ui and chest_ui.chest:
4         var success = chest_ui.chest.insert(slot_data.item)
5         if success:
6             slot_data.amount -= 1
7             if slot_data.amount <= 0:
8                 slot_data.item = null

```

Kod 25: Przykład: przenoszenie do skrzyni

4.9.3.5 Synchronizacja UI

Podobnie jak w przypadku ekwipunku, UI skrzyni aktualizuje się na podstawie sygnału `update_slots`, emitowanego przy każdej modyfikacji zawartości. Funkcja `update_slots()` przypisuje dane ze sceny do komponentów wizualnych.

4.9.3.6 Efekty dźwiękowe i animacje

Otwieraniu i zamykaniu skrzyni towarzyszą animacje i dźwięki (`ChestOpen`, `ChestColse`), co wzmacnia odbiór interakcji i wprowadza przyjemne efekty estetyczne.

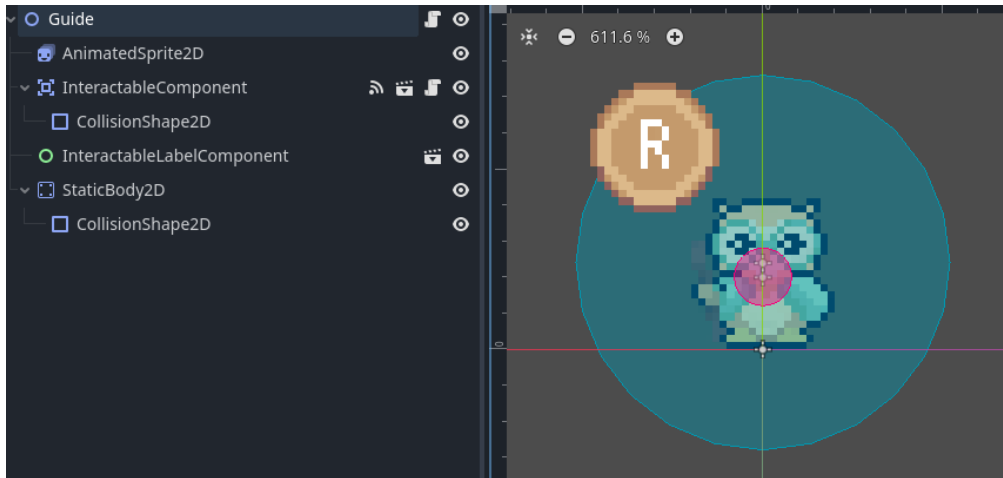
4.10 System samouczka

System samouczka w grze *Bloomrise* został zrealizowany przy użyciu rozszerzenia **Dialogue Manager** dla silnika Godot. Odpowiada on za prezentację konwersacji z przewodnikiem (sową Wisowl) w formie okien dialogowych z reakcjami i wyborami gracza, a także za mutacje stanu gry w czasie rzeczywistym.

4.10.1 Uruchamianie samouczka i interakcja z przewodnikiem

Samouczek uruchamiany jest poprzez interakcję z postacią przewodnika – sową Wisowl. W świecie gry postać ta reprezentowana jest przez scenę `Guide.tscn`, która zawiera m.in. komponenty kolizji oraz system interakcji:

- `AnimatedSprite2D` – odpowiada za animację sowy,
- `InteractableComponent` – definiuje obszar interakcji wokół postaci (`Area2D`),
- `InteractableLabelComponent` – odpowiada za wyświetlenie etykiety z ikoną klawisza (np. `R`),
- `StaticBody2D` – zapewnia kolizję fizyczną w świecie gry.



Rysunek 19: Struktura przewodnika (Guide.tscn) — postać sowy z obszarem interakcji i etykietą aktywną.

Aby rozpocząć samouczek, gracz musi podejść do sowy i wcisnąć klawisz przypisany do akcji `show_dialogue` (domyślnie R). Interakcja ta wyzwała otwarcie pierwszego wpisu dialogowego z przypisanego pliku `guide.dialogue` i rozpoczyna cały proces konwersacji oraz przydzielania narzędzi i zadań.

Mechanizm działania `InteractableComponent` bazuje na wykrywaniu obecności gracza w obszarze kolizyjnym i aktywacji sygnału wejścia:

```
1 signal interactable_activated
2 signal interactable_deactivated
3
4 func _on_body_entered(body: Node2D) -> void:
5     interactable_activated.emit()
6
7 func _on_body_exited(body: Node2D) -> void:
8     interactable_deactivated.emit()
```

Kod 26: Obsługa aktywacji interakcji w `InteractableComponent`

Po aktywacji, na ekranie pojawia się ikonka przycisku (np. „R”), a naciśnięcie odpowiedniego klawisza powoduje uruchomienie balonika dialogowego i rozpoczęcie samouczka.

```
1 if in_range and Input.is_action_just_pressed("show_dialogue"):
2     DialogueManager.show_balloon(dialogue_resource)
```

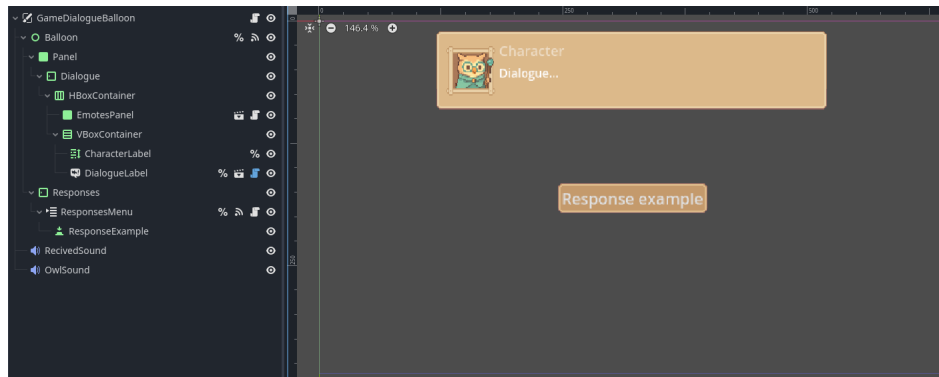
Kod 27: Wywołanie dialogu przez gracza

4.10.2 Struktura hierarchii UI

W scenie `GameDialogueBalloon.tscn` znajdują się główne komponenty interfejsu:

- `Balloon (Control)` – główny kontener dialogowy,
- `Panel` → `Dialogue` – zawiera treść dialogu,
- `HBoxContainer` → `EmotesPanel` – odpowiada za animowane emotki postaci,
- `VBoxContainer`:
 - `CharacterLabel` – pokazuje imię mówiącej postaci,
 - `DialogueLabel` – wyświetla treść dialogu z efektem pisanego,
- `Responses` → `ResponsesMenu` – wyświetla możliwe odpowiedzi gracza,

- `ResponseExample` – przykład przycisku odpowiedzi.



Rysunek 20: Interfejs dialogowy z sową Wisowl — głównym przewodnikiem samouczka.

4.10.3 Skrypt `game_dialogue_balloon.gd` – logika działania

Klasa dziedziczy po `BaseGameDialogueBalloon` i zarządza przebiegiem samouczka oraz integracją z innymi systemami gry.

`start()` Rozpoczyna dialog, pobiera jego pierwszy wpis z zasobu `DialogueResource`.

`apply_dialogue_line()` Odpowiada za:

- pokazanie interfejsu z tekstem dialogu,
- wykonanie efektu „pisania” tekstu,
- obsługę mutacji (`@mutation`), np. przyznanie przedmiotu,
- pokazanie dostępnych odpowiedzi gracza (jeśli są).

`next(next_id: String)` Przechodzi do kolejnego wersu dialogu.

`_on_mutated()` Obsługuje instrukcje samouczka (`@mutation`), takie jak:

- przyznawanie narzędzi (`tool_pickaxe`, `tool_axe`, itd.),
- ustawianie punktów kontrolnych (`checkpoint_after_gathering`, itd.),
- dodawanie zadań (`add_tasks`),
- odtwarzanie dźwięków (`recived_sound.play()`).

Gdy pojawi się mutacja, system wykonuje odpowiednie akcje zdefiniowane w nadpisanej metodzie `_on_mutated()` w klasie `game_dialogue_balloon.gd`. Przykładowo, nadanie graczowi kilofa wygląda tak:

```
1 if mutation.has("tool_pickaxe"):
2     var item = InventoryManager.item_db["Pickaxe"]
3     ToolsPanel.add_item_to_first_free_slot(item)
```

Kod 28: Dodanie kilofa do ekwipunku gracza

`_on_balloon_gui_input()` Reaguje na kliknięcia lub skróty klawiaturowe – umożliwia pominięcie animacji pisania lub przejście dalej.

4.10.4 Plik dialogowy – struktura i działanie

Plik dialogowy (`guide.dialogue`) jest tekstowym scenariuszem opisującym przebieg rozmowy z graczem. Składa się z nazwanych bloków (np. `tutorial`, `skip_tutorial`), które zawierają:

- kwestie dialogowe postaci (np. sowy Wisowl),
- instrukcje kontrolne, takie jak `@mutation`,
- opcje odpowiedzi dla gracza i przejścia do kolejnych bloków.

Struktura pliku przypomina prosty język skryptowy, który definiuje logikę samouczka jako rozgałęzioną narrację. Każdy blok zaczyna się od identyfikatora poprzedzonego tyldą i może zawierać wiele linii wypowiedzi, opcje wyboru oraz polecenia modyfikujące stan gry.

Przykład użycia:

```
~ tutorial
Wisowl: First things first | let's gather some basic resources!
@mutation tool_pickaxe
@mutation tool_axe
=> checkpoint_after_gathering
```

Powyższy fragment:

1. wyświetla tekst wypowiedzi przewodnika,
2. wykonuje mutacje: dodaje graczowi kilof i siekierę,
3. kończy blok, przechodząc do identyfikatora `checkpoint_after_gathering`.

Opcje odpowiedzi są definiowane jako lista z wyborem gracza i powiązanym następnym krokiem:

```
- Yes, please. => tutorial
- No, I already know everything. => skip_tutorial
```

Taki mechanizm pozwala tworzyć interaktywne, warunkowe scenariusze, w których gracz podejmuje decyzje wpływające na przebieg samouczka. Dzięki zastosowaniu mutacji, plik dialogowy nie tylko prowadzi narrację, ale także bezpośrednio steruje logiką gry (np. przydzielaniem narzędzi, aktywowaniem zadań, zmianą punktów kontrolnych).

4.10.5 GameState i mechanizm pauz w samouczku

Aby umożliwić graczowi wykonywanie czynności opisanych przez sowę Wisowl w swoim tempie, system samouczka wykorzystuje mechanizm **punktów kontrolnych (checkpointów)** oraz zmienną `GameState.guide_checkpoint`.

Każdy etap samouczka (np. zbieranie zasobów, obsługa ekwipunku, sadzenie) kończy się tzw. pauzą, czyli chwilą, gdy dialog zostaje wstrzymany, a gracz ma czas na wykonanie przedstawionej akcji. Przykłady takich punktów to:

- `checkpoint_after_gathering` – po zebraniu drewna i kamieni,
- `checkpoint_after_inv` – po zapoznaniu się z ekwipunkiem,
- `checkpoint_after_planting` – po zasadzeniu nasion,
- `checkpoint_after_watering` – po podlaniu roślin.

Stan samouczka zapisywany jest w zmiennej `GameState.guide_checkpoint`, co pozwala systemowi dialogowemu:

- wznowić samouczek od właściwego miejsca po przerwie lub restarcie,

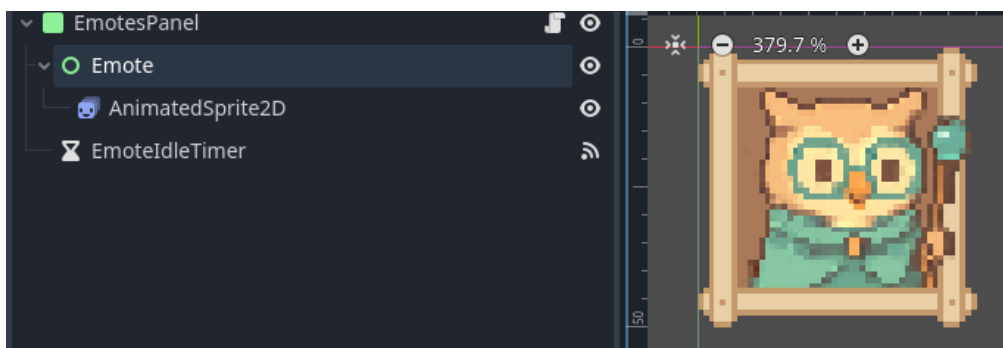
- dynamicznie reagować na postępy gracza (np. dopiero gdy zakończy sadzenie, pojawi się dalsza część rozmowy),
- zatrzymać narrację i dać czas na działanie, zamiast narzucać sztywne tempo.

Ten system zwiększa interaktywność i komfort użytkownika, ponieważ pozwala mu przyswajać nowe informacje i testować mechaniki w praktyce, bez presji i przerywania dialogiem.

4.10.6 Elementy wizualne i animacje

W celu zwiększenia immersji i atrakcyjności samouczka, system dialogowy wzbogacony został o elementy wizualne i dźwiękowe, które urozmaicają interakcję gracza z sową Wisowl.

- **EmotesPanel** – specjalny komponent odpowiedzialny za mimikę i gesty postaci. Wewnątrz znajduje się węzeł **AnimatedSprite2D**, który odtwarza przypisane animacje:
 - animacje domyślne (np. `emote_1_idle`) są uruchamiane automatycznie na starcie sceny,
 - losowe reakcje emocjonalne (np. `emote_3_talk`, `emote_2_advice`) są dobierane dynamicznie przy każdej nowej linijce dialogu,
 - timer `EmoteIdleTimer` okresowo przywraca animacje stanu spoczynku.



Rysunek 21: Struktura EmotesPanel pozwalającego na animację sowy przewodnik Wisowl

- **Efekt pisania tekstu** – za pomocą `DialogueLabel` każdy dialog wyświetlany jest z opóźnieniem literowym („typing effect”), który symuluje naturalne tempo wypowiedzi postaci. Gracz może pominąć animację klikając LPM lub naciskając przypisany przycisk (`skip_action`).
- **Dźwięki** – system wykorzystuje dwa dedykowane węzły `AudioStreamPlayer2D`:
 - `OwlSound` – odtwarzany przy każdej wypowiedzi Wisowl, co nadaje jej charakterystyczny głos,
 - `ReceivedSound` – uruchamiany przy każdej mutacji (`@mutation`), co daje graczowi wyraźną informację o dodaniu przedmiotów do jego ekwipunku.
- **Reakcja na wybory** – przy każdej zmianie linijki dialogu lub wyborze odpowiedzi uruchamiana jest nowa animacja/emocja sowy, co dodatkowo wzmacnia efekt interaktywności i „żywości” postaci.

Dzięki powyższym mechanizmom system dialogowy nie jest tylko prostym oknem tekstowym, ale pełnoprawnym komponentem narracyjnym z zachowaniem rytmu, ekspresji i interaktywności — co znacząco wpływa na odbiór gry i przyswajanie informacji przez gracza.

4.11 System zadań

4.11.1 Struktura

System zadań oparty jest na singletonie `TaskManager.gd`, który zarządza kolejką celów do wykonania przez gracza. W danym momencie aktywne może być tylko jedno zadanie, reprezentowane przez zmienną `current_task`. Pozostałe zadania przechowywane są w kolejce `task_queue`.

Każde zadanie opisane jest jako słownik zawierający następujące pola:

```
{
    "name": "Collect Stones",
    "id": "stone",
    "target": 10,
    "progress": 3,
    "award": 5
}
```

- `id` – unikalny identyfikator zasobu (np. "stone", "plant_rose"),
- `target` – liczba wymaganych akcji do ukończenia,
- `progress` – aktualny postęp gracza,
- `award` – nagroda za wykonanie zadania (np. monety).

4.11.2 Wybrane funkcje systemu

System udostępnia szereg metod do zarządzania zadaniami. Poniżej opisano najważniejsze z nich:

`add_task()` Dodaje nowe zadanie do kolejki. Jeśli brak aktywnego zadania, ustawia je jako bieżące:

```
1 func add_task(name: String, id: String, target: int, award: int = 10) -> void:
2     task_queue.append({ ... })
3     if current_task.is_empty():
4         _next_task()
```

`report_progress()` Rejestruje postęp w wykonaniu zadania:

```
1 func report_progress(resource_id: String, amount: int = 1) -> void:
2     if current_task["id"] == resource_id.to_lower():
3         current_task["progress"] += amount
4         if current_task["progress"] >= current_task["target"]:
5             _on_task_completed()
```

`_on_task_completed()` Obsługuje zakończenie zadania — przyznaje nagrodę, odtwarza dźwięk i uruchamia kolejne zadanie:

```
1 func _on_task_completed():
2     GameManager.add_gold(current_task["award"])
3     task_complete_sound.play()
4     _next_task()
```

4.11.3 Kolejność i aktualizacja

1. Zadania są dodawane przez `add_task()`.
2. System utrzymuje kolejkę — tylko jedno zadanie może być aktywne jednocześnie.
3. Funkcja `report_progress()` aktualizuje postęp na podstawie wykonanego działania.
4. Po osiągnięciu celu:

- wywoływana jest funkcja `_on_task_completed()`,
- uruchamiany jest dźwięk sukcesu,
- przyznawana jest nagroda (złoto),
- emitowany jest sygnał `tasks_updated`,
- inicjowane jest kolejne zadanie.

4.11.4 Interfejs użytkownika

Za prezentację zadań na ekranie odpowiada skrypt `TaskUI.gd`. Interfejs ten działa w oparciu o komponent `RichTextLabel`, znajdujący się w hierarchii UI gry (np. `$Panel/TaskListLabel`), który wyświetla aktualny status zadania.

System nasłuchuje sygnału `tasks_updated`, emitowanego przez `TaskManager` zawsze, gdy zadanie zostaje zaktualizowane, rozpoczęte lub zakończone.

Funkcja `update_task_list()` pobiera treść zadania z funkcji `get_task_descriptions()` i aktualizuje widoczny tekst. Przykładowy format wyświetlanych zadań:

- `Collect Logs 2/3 (Reward: 5 coins)` – zadanie w toku z podaną nagrodą,
- `Currently you have no active tasks.` – brak aktywnego zadania.

Jeśli nagroda (`award`) wynosi 0, informacja o niej nie jest wyświetlana, aby nie zaśmiecać interfejsu.

Poniżej fragment kodu odpowiedzialnego za aktualizację widoku:

```
1 func update_task_list():
2     var lines = TaskManager.get_task_descriptions()
3     if lines.is_empty():
4         task_list_label.text = "Currently you have no active tasks."
5     else:
6         task_list_label.text = "Tasks:\n" + "\n".join(lines)
```

4.11.5 Integracja z grą

System zadań zintegrowany jest z wieloma komponentami gry:

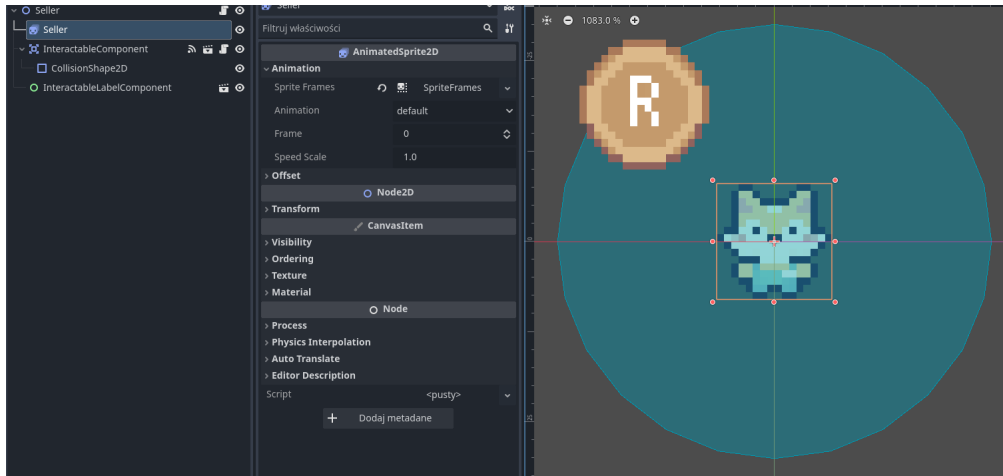
- Zadania mogą być inicjowane przez NPC (np. `guide.gd`),
- Mogą być aktywowane przez komponenty interaktywne (np. `CollectableComponent`),
- System wspiera różne ścieżki – gracz może przejść tutorial lub go pominąć, a gra dynamicznie dostosowuje zestaw zadań.

4.12 System działania sklepu

System sklepu umożliwia graczowi kupowanie i sprzedawanie nasion roślin w oparciu o interfejs graficzny oraz aktualną ilość posiadanego złota. Składa się z dwóch głównych sekcji: zakupu (`BuyTab`) i sprzedaży (`SellTab`). W każdej z nich wykorzystywana jest prefabrykowana scena `ShopItem`, która dynamicznie prezentuje dane danego przedmiotu oraz przyciski umożliwiające interakcję. Aktualnie w sklepie sprzedawane są tylko nasiona, ale może być łatwo rozbudowany o nowe towary. Do zakupu możliwe są nasiona pobrane z przygotowanej listy, do sprzedaży dynamicznie pobierana jest lista przedmiotów dostępnych w ekwipunku. Pobrane przedmioty wstawiane są do elementu `ShopItem`, który formatuje otrzymane dane.

4.12.1 Obecność w grze

Sklep aktywowany jest w trakcie gry poprzez interakcję z dedykowanym NPC – analogicznie jak w przypadku samouczka z sową Wisowl. Gracz musi podejść do sprzedawcy, a następnie nacisnąć przypisany przycisk interakcji (domyślnie R, powiązany z akcją `show_dialogue`). W wyniku tego wywoływana jest logika otwierająca interfejs sklepu i uruchamiająca dialog ze sprzedawcą.



Rysunek 22: Struktura sceny NPC sprzedawcy. Obiekt zawiera komponent kolizyjny, system interakcji (*InteractableComponent*) oraz etykietę informacyjną (*InteractableLabelComponent*) z ikoną klawisza R. Podejście gracza i naciśnięcie przycisku wywołuje interfejs sklepu.

4.12.2 Struktura UI

Sklep jest reprezentowany jako *CanvasLayer*, co zapewnia, że jego GUI znajduje się zawsze na wierzchu sceny. Główne komponenty to:

- 'TabContainer' — przełącza widok między zakładkami kupna i sprzedaży,
- 'item_grid' — kontener *GridContainer*, do którego dodawane są przyciski zakupu,
- 'sell_grid' — analogiczny kontener do prezentacji przedmiotów do sprzedaży,
- 'gold_label' — etykieta wyświetlająca aktualną ilość złota gracza.



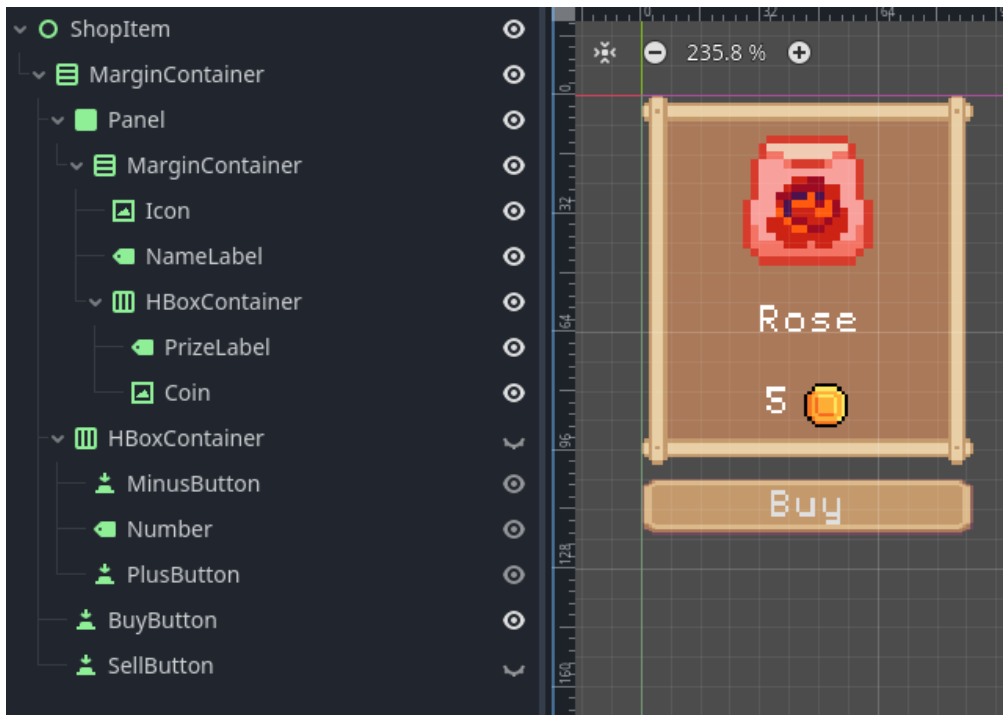
Rysunek 23: Interfejs sklepu.

4.12.2.1 Tworzenie przedmiotów do kupienia

Podczas inicjalizacji sceny, metoda `_create_buy_items()` tworzy dynamicznie przyciski zakupowe na podstawie listy 'item_data'. Dla każdego wpisu:

- ustawiany jest odpowiedni obraz (*AtlasTexture*) z arkusza `flowers.png`,
- przypisywana jest nazwa i cena nasiona,

- podpinana jest funkcja obsługująca zakup.



Rysunek 24: Interfejs sceny przedmiotu.

```

1 func _create_buy_items():
2     var item_data = [
3         {"name": "Red Rose", "price": 10, "icon_index": 0},
4         ...
5     ]
6     for item in item_data:
7         var item_instance = ShopItemScene.instantiate()
8         var buy_button = item_instance.get_node("../BuyButton")
9         buy_button.pressed.connect(func(): _buy_item(item))
10        item_grid.add_child(item_instance)

```

Kod 29: Tworzenie scen przedmiotów

4.12.2.2 Formatowanie i mapowanie nazw

Ze względu na różnice między nazwami przedmiotów wyświetlanymi graczowi a ich nazwami wewnętrznymi, używana jest mapa `name_dictionary`, która konwertuje np. "Red Rose" na "RedRoseSeeds". Dodatkowo `format_name()` wstawia spacje między członami nazw, by ułatwić czytelność w GUI.

4.12.2.3 Obsługa dźwięków i aktualizacji złota

Każda operacja finansowa (kupno/sprzedaż) powoduje:

- odtworzenie przypisanego dźwięku (`buy_sound` lub `coin_recived_sound`),
- aktualizację etykiety złota poprzez `update_gold()`.

4.12.3 System sprzedaży przedmiotów

Podczas zakupu system sprawdza, czy gracz posiada wystarczającą ilość złota. Jeśli tak:

- odejmowana jest odpowiednia kwota,
- gracz otrzymuje przedmiot do ekwipunku (`InventoryManager.add_collectable()`),
- odtwarzany jest dźwięk zakupu.

```

1 func _buy_item(item):
2     if GameManager.player_gold >= item.price:
3         GameManager.subtract_gold(item.price)
4         buy_sound.play()
5         InventoryManager.add_collectable(get_base_name(item.name))
6     else:
7         print("Not enough gold!")

```

Kod 30: Funkcja zakupu

W zakładce SellTab gracz może sprzedać posiadane nasiona lub rośliny. Każdy slot z możliwym do sprzedania przedmiotem generuje osobny ShopItem z przyciskiem SellButton oraz kontrolkami zmiany ilości.

```

1 func _add_sell_item(slot):
2     var count := int(slot.amount)
3     sell_button.pressed.connect(func():
4         if InventoryManager.remove_item(slot.item, count):
5             GameManager.add_gold(price * count)
6             populate_sell_items()

```

Kod 31: Obsługa sprzedaży przedmiotów

4.12.3.1 Integracja

System sklepu został zaprojektowany modularnie i umożliwia:

- łatwe dodawanie nowych przedmiotów,
- rozszerzenie o inne kategorie zakupowe (np. narzędzia),
- integrację z systemem misji (`TaskManager.report_progress()`).

5 Opis napotkanych problemów

5.1 Dźwięk podczas zbierania collectables

Podczas dodawania efektów dźwiękowych próbowaliśmy dodać dźwięk podczas zbierania elementów ‘collectable’. Został on dodany do sceny `Collectable_component` i uruchomiony podczas wywoływania funkcji podczas kolizji z postacią gracza.

```

1 func _on_body_entered(body: Node2D) -> void:
2     if body is Player:
3         var was_collected = InventoryManager.add_collectable(collectable_name)
4         if was_collected:
5             CollectSound.play()
6             print("Collected: ", collectable_name)
7             get_parent().queue_free()
8         else:
9             print("Could not collect:", collectable_name)

```

Kod 32: Pierwotna funkcja odtwarzania dźwięku zbierania instancji scen zawierających komponent collectable

Jednak ten dźwięk nie był odtwarzany, ponieważ scena ‘collectable’ jest w tej samej funkcji usuwana. Aby rozwiązać ten problem utworzyliśmy globalny skrypt `Scene_manager`, do którego dodałyśmy funkcję odtwarzającą odpowiedni dźwięk.

```

1 @onready var collect_sound := preload("res://assetss/Sounds/collect-points-190037.mp3")
2
3 func play_collect_sound():
4     var sound = AudioStreamPlayer.new()
5     sound.stream = collect_sound
6     add_child(sound)
7     sound.play()
8     sound.finished.connect(sound.queue_free)

```

Kod 33: Globalna funkcja dla dźwięku zbierania

W scenie ‘collectable’ ta funkcja została wykorzystana zamiast odwołania do wewnętrznego `AudioStreamPlayer`.

```

1 SceneManager.play_collect_sound()

```

Kod 34: Poprawione odtwarzanie dźwięku

5.2 Wyświetlanie elementów

Podczas testowania projektu zaobserwowano problem z niepoprawnym wyświetlaniem postaci gracza i innych obiektów na scenie. Elementy nakładały się na siebie w sposób nielogiczny: np. gracz „wchodził” na ściany budynku, które powinny go przysłaniać, lub znajdował się nad roślinami, które rosły przed nim. Problem ten wynikał z braku prawidłowego ustalania kolejności renderowania elementów 2D.

Y-Sorting – rozwiązanie problemu z kolejnością rysowania

W grafice 2D w grach typu top-down (lub z rzutem izometrycznym) naturalnym rozwiązaniem problemu kolejności wyświetlania obiektów jest technika **Y-sorting**. Polega ona na sortowaniu obiektów względem ich współrzędnej Y — im niżej obiekt znajduje się na ekranie (czyli im większa wartość Y), tym powinien znajdować się bliżej gracza i być narysowany „na wierzchu”. Odwrotnie – obiekty znajdujące się wyżej (mniejsza wartość Y) powinny być rysowane jako „z tyłu”.

W silniku **Godot** można to uzyskać poprzez użycie węzła `YSort`, który automatycznie sortuje wszystkich swoich potomków według pozycji Y.

Problemy po aktywacji YSort

Początkowo, po dodaniu węzła `YSort` i umieszczeniu w nim odpowiednich elementów, nadal występowały problemy z kolejnością rysowania. Obiekty w dalszym ciągu nie wyświetlały się w logiczny sposób. Powodem był nieodpowiedni układ sceny: część obiektów (np. drzewa, gracz, przedmioty) znajdowała się poza strukturą `YSort`, co skutkowało brakiem wspólnego kontekstu sortowania.

Ostateczne rozwiązanie

Aby poprawnie zastosować sortowanie względem osi Y:

- Utworzono jeden główny węzeł typu `YSort` w głównej scenie gry.
- Wszystkie obiekty, które powinny być sortowane względem siebie (gracz, NPC, drzewa, budynki, interaktywne obiekty) zostały przeniesione jako potomkowie tego węzła.
- Obiekty statyczne, które nie podlegają sortowaniu (np. tło, warstwy dekoracyjne, nieinteraktywne tile-mapy), pozostały poza węzłem `YSort`, aby nie wpływały na kolejność rysowania.

Dodatkowo upewniono się, że każdy z obiektów ma poprawnie ustawioną pozycję odniesienia — w przypadku sprite’ów często należy dostosować środek sprite’a (pivot), aby pozycja `position.y` odpowiadała rzeczywistej „podstawie” obiektu (np. stóp gracza lub podstawy drzewa).

Efekt końcowy

Dzięki zastosowaniu węzła `YSort` i odpowiedniemu uporządkowaniu hierarchii sceny:

- gracz może wchodzić „za” drzewa, domy i inne wysokie obiekty, zachowując naturalne wrażenie głębi,
- animacje i interakcje (np. drzwi, ognisko, zbierane rośliny) są wyświetlane w spójnej kolejności względem postaci,
- obraz gry wygląda bardziej profesjonalnie i czytelnie.

Technika Y-sorting znacząco poprawia wrażenia wizualne i jest powszechnie stosowana w grach 2D z widokiem z góry lub w rzucie ukośnym.

5.3 Problemy z integracją ekwipunku i panelu narzędzi

Na początku system ekwipunku oraz paska narzędzi były implementowane jako niezależne moduły. Sloty w panelu ekwipunku (`Item_slot`) oraz w pasku narzędzi (`Tool_slot`) opierały się na różnych typach danych i logice działania, co prowadziło do następujących problemów:

- brak kompatybilności typów – dane w ekwipunku oparte były o strukturę `InvSlot`, podczas gdy pasek narzędzi przyjmował tylko bezpośrednie odniesienia do obiektu `InvItem`,
- powielanie logiki – funkcje odpowiedzialne za aktualizację UI i obsługę kliknięć były duplikowane w obu systemach,
- trudności w przesyłaniu danych – np. sygnał `item_sent_to_toolbar` wymagał dodatkowej logiki transformującej strukturę danych,
- błędy przy próbie dodania narzędzi do toolbaru – brak walidacji, brak reakcji wizualnej lub niespójne stany UI.

5.3.1 Refaktoryzacja struktury danych

Rozwiązaniem tych problemów było ujednolicenie struktury danych poprzez pełne oparcie się na zasobie `InvSlot`. Zamiast bezpośrednio operować na obiektach typu `InvItem`, zarówno sloty ekwipunku, jak i toolbaru zaczęły wykorzystywać wspólną metodę `set_item(item)` oraz sygnały przekazujące referencję do slotu, nie tylko samego przedmiotu.

5.3.2 Efekty wprowadzenia jednolitego modelu

Dzięki temu:

- możliwa była pełna integracja systemów – przenoszenie przedmiotów między ekwipunkiem a toolbar działało płynnie,
- logika związana z aktualizacją ikony i liczby przedmiotów została uproszczona i scentralizowana,
- interakcje użytkownika (LPM, PPM, Shift+LPM) działały spójnie w całym systemie,
- system stał się łatwiejszy do rozbudowy o nowe funkcjonalności (np. przeciąganie przedmiotów myszką).

To doświadczenie pokazało, jak istotne jest planowanie spójnych typów danych i wspólnego interfejsu komunikacji między komponentami UI, nawet jeśli początkowo wydają się niezależne.

6 Podsumowanie

Projekt „*Bloomrise*” umożliwił praktyczne wykorzystanie różnorodnych umiejętności związanych z projektowaniem i programowaniem gier w silniku Godot Engine. Stanowił okazję do pogłębienia wiedzy z zakresu zarówno warstwy technicznej (m.in. architektura kodu, logika gry, integracja komponentów), jak i projektowej (np. projektowanie rozgrywki, user experience, balans mechanik). W trakcie pracy miałyśmy szansę zmierzyć się z realnymi wyzwaniami, jakie towarzyszą produkcji gry — od planowania funkcjonalności, przez implementację, po testowanie i dopracowywanie szczegółów.

W ramach realizacji powstała w pełni funkcjonalna gra 2D, łącząca elementy zarządzania zasobami, uprawy roślin, eksploracji otwartego świata i ekonomii opartej na handlu, z rozbudowanym i spójnym interfejsem użytkownika. *Bloomrise* wyróżnia się nie tylko spójną stylistyką graficzną, ale też licznymi systemami wzajemnie się uzupełniającymi, tworzącymi wciągający i satysfakcjonujący cykl rozgrywki.

Na poziomie projektowym, gra została zaplanowana jako otwarta eksploracyjnie przestrzeń, w której gracz — wcielając się w postać kota-ogrodnika — może rozwijać własny ogród, korzystać z różnorodnych narzędzi, prowadzić uprawę roślin i handlować zasobami. Gra oferuje również system samouczka, który ułatwia wejście w świat rozgrywki, oraz system zadań prowadzący gracza przez kolejne cele.

Od strony technicznej, szczególny nacisk został położony na modularność i czystość kodu. Wykorzystanie maszyny stanów, systemów singletonów, a także komponentowego podejścia do interakcji i logiki gry, umożliwiło wysoką elastyczność oraz łatwość rozbudowy projektu. Implementacja kluczowych systemów, takich jak: system pogody, cykl dobowy, rozwój roślin, użycie narzędzi, ekwipunek, sklep, zadania i samouczek – pokazuje szerokie spektrum zastosowanych rozwiązań architektonicznych.

Szczególną uwagę poświęcono detalom, które wpływają na komfort i immersję gracza. Dodano m.in. dźwięki przy zbieraniu przedmiotów, wyborze narzędzi czy interakcji ze środowiskiem, co wzbogaca doświadczenie rozgrywki. Zastosowano także animacje takie jak trzęsące się drzewo podczas ścinania, animacje nawadniania roślin czy delikatne poruszenia postaci sowy w samouczku – w tym jej animowane „milczenie” po dłuższej chwili mówienia. Tego rodzaju dopracowanie drobnych elementów znacząco wpłynęło na odbiór gry jako dopracowanej, przyjemnej i przemyślanej produkcji.

Efektem końcowym jest gra z przejrzystym interfejsem, bogatą interaktywnością oraz estetyką przyjazną graczowi. Projekt nie tylko spełnia założone cele dydaktyczne, ale również może służyć jako solidna baza do dalszej rozbudowy o nowe funkcje, takie jak system zwierząt, sieć społecznościowa NPC czy bardziej rozbudowana ekonomia.

Projekt ten stanowi wartościowe doświadczenie zarówno pod względem praktycznym, jak i koncepcyjnym. Umożliwił poznanie i zastosowanie pełnego cyklu tworzenia gry: od pomysłu, przez projektowanie mechanik, implementację, testowanie, aż po końcową prezentację gotowego produktu. W rezultacie powstała spójna, wciągająca i technicznie dopracowana produkcja, która może być wizytówką umiejętności zdobytych w trakcie studiów.

7 Źródła

1. DevWorm. *A Complete Guide to Y-SORTING in Godot 4.3+*. YouTube, 2024.
https://www.youtube.com/watch?v=lvuLjMAr_BE
2. RapidVectors. *How to Build a Complete 2D Farming Game – an 8-Hour Tutorial Series – Godot 4.3*. YouTube, 2024.
<https://www.youtube.com/watch?v=it0lsREGdmc>
3. DevWorm. *Create a Complete Survival Game in Godot 4 (Step by Step)*. YouTube, 2023.
https://www.youtube.com/watch?v=eAEe_9jCV4s
4. Nesi. *How to Make The Smoothest Pixel Art Camera in Godot*. YouTube, 2022.
<https://www.youtube.com/watch?v=LoR4Xg1129U>
5. *Dialogue Manager – Godot Asset Library*. Godot Engine.
<https://godotengine.org/asset-library/asset/1207>
6. *Oficjalna dokumentacja silnika Godot 4.x*.
<https://docs.godotengine.org/pl/4.x/>
7. *Free Game Assets – itch.io*.
<https://itch.io/game-assets/free>
8. *CraftPix – Game Assets and Graphic Resources*.
<https://craftpix.net/>
9. *Kenney – Free Game Assets*.
<https://kenney.nl/assets>
10. *Mixkit – Free Sound Assets*.
<https://mixkit.co/free-sound-effects/>
11. *Pixabay – Royalty-free Sound Effects*.
<https://pixabay.com/sound-effects/>